

1. Ingest Data with Microsoft Fabric

Ingest Data with Dataflows Gen2

Unit 1: Dataflows Gen2

Core Definition: A low-code, cloud-based tool used for **ETL** (Extract, Transform, Load) using the visual Power Query Online interface.

Best Use Cases:

- **Reusable ETL Logic:** Prep data once for multiple users.
- **Curated Semantic Models:** Prepare data specifically for Power BI.
- **Standardization:** Creating universal tables (like a Master Date Table).

The Workflow:

1. **Connect** (Source) → 2. **Transform** (Clean) → 3. **Land** (Destination)

Comparison: Pros vs. Cons

Strengths	Weaknesses
Consistency: Promotes "extract once, reuse many times."	Not a Warehouse: For prep, not long-term storage.
Performance: Reduces load on source systems.	No RLS: Does not support row-level security.
Accessibility: Low-code for non-engineers.	Requirements: Needs a Fabric capacity workspace.

Unit 3: Exploring the Dataflows Gen2 Interface

Core Definition: Dataflows Gen2 uses Power Query Online, providing a visual environment to build data transformations without writing code, while the M language runs in the background.

 The 5 Key Interface Areas

Area	Purpose	Key Features

1. Power Query Ribbon	The "Toolbox"	Access connectors, transformations (Pivot/Unpivot), and set the Data Destination .
2. Queries Pane	The "Navigator"	Lists all data sources. Use Duplicate or Reference to create Star Schemas.
3. Diagram View	The "Map"	A visual flowchart showing how data flows from source to transformation steps.
4. Data Preview	The "Sample"	Shows a subset of data to preview how transformations affect the results.
5. Query Settings	The "History"	Lists Applied Steps . Allows you to rename, reorder, or delete specific transformations.

Key Capabilities

- **Data Destinations:** Land data directly in a **Lakehouse**, **Warehouse**, or **SQL Database**.
- **Advanced Editor:** Access and edit the underlying **M code** for complex logic.
- **Query Folding:** Pushes the transformation work back to the source database for significantly faster performance.

 **Pro-Tip for Review:** Use the **Reference** option in the Queries Pane when you need to use one source for multiple tables. It avoids loading the raw data multiple times and keeps your workflow efficient!

Unit 4: Orchestrating with Data Pipelines

Core Definition: While a **Dataflow** is the "worker" that cleans data, a **Data Pipeline** is the "manager" that schedules, triggers, and connects multiple workers together into a single automated workflow.

The Role of Pipelines

Pipelines provide a visual way to chain activities in a specific order. You use them when your process requires more than just a simple data load.

Common Pipeline Activities:

- **Incorporate Dataflow:** Trigger a Dataflow Gen2 to refresh and land data.

- **Copy Data:** Move large volumes of data quickly between sources.
- **Add Notebook:** Run complex Python/Spark code.
- **Execute Script:** Run SQL stored procedures or scripts after the data lands.
- **Get Metadata:** Check if a file exists before trying to process it.

⚡ Why Combine Dataflows + Pipelines?

1. **Automation:** Instead of manual refreshes, use **Schedules** or **Triggers** (e.g., run every morning at 8 AM).
2. **Sequential Logic:** Ensure the "Clean Data" step (Dataflow) finishes successfully before the "Update Report" step (Script) begins.
3. **Error Handling:** Set up notifications if a Dataflow fails so you can fix it immediately.

⚖️ Summary: Worker vs. Manager

Feature	Dataflow Gen2 (The Worker)	Data Pipeline (The Manager)
Main Goal	Transforming & cleaning data.	Orchestrating & scheduling tasks.
Interface	Power Query (Low-code).	Pipeline Canvas (Drag-and-drop).
Logic	Data-centric (Joins, Pivots).	Task-centric (Wait, If/Else, Loops).
Capability	Can land data in a Lakehouse.	Can trigger Dataflows, Notebooks, & Scripts.

💡 **Pro-Tip for Review:** For the exam, remember that **Dataflows** are best for *transforming* data, but **Pipelines** are required for *automation* and *complex sequencing*. Always "wrap" your Dataflow in a Pipeline for enterprise-grade solutions.

Orchestrate processes with Pipelines

Unit 1 Introduction

Definition: A data pipeline is a defined sequence of activities used to orchestrate an overall process—primarily extracting data from sources and loading it into a destination.

Core Function (ETL): Pipelines are most commonly used to automate **Extract, Transform, and Load (ETL)** processes. This involves taking transactional data from operational stores and moving it into analytical stores like a **lakehouse, data warehouse, or SQL database**.

Architecture: If you are familiar with **Azure Data Factory**, the architecture in Fabric will be identical. It uses connected activities to manage data processing tasks and control flow logic.

Execution: Pipelines can be initiated manually (interactively) through the user interface or set to run on a specific schedule.

1. Activities: The Building Blocks

Activities are the individual steps in your workflow. They are divided into two main types:

- **Data Transformation Activities:** These do the "heavy lifting" of moving or changing data.
 - **Copy Data:** The most common task—extracting from a source and loading to a destination.
 - **Notebook / Stored Procedure:** Running custom Spark or SQL code for complex logic.
 - **Data Flow (Gen2):** Using a Power Query-like interface for visual transformations.
- **Control Flow Activities:** These act as the "logic" or "brain" of the pipeline.
 - **Loops & Conditionals:** Creating "If" statements or "For Each" loops to repeat tasks.
 - **Variables/Parameters:** Storing and updating values during the run.

2. Parameters: Flexibility & Reusability

Instead of creating a new pipeline for every single folder or file, you use **Parameters**.

Example: You create one pipeline to move "Sales Data," but use a parameter to specify the **Month**. This allows you to reuse the same pipeline for January, February, and March just by changing the input value at runtime.

3. Pipeline Runs

A "run" is a specific execution of your pipeline.

- **On-Demand:** You trigger it manually (often during testing).
- **Scheduled:** You set it to run automatically (e.g., every night at 2:00 AM).

- **Monitoring:** Every run gets a **unique Run ID**, which you use to check if the process succeeded or failed and to troubleshoot errors.

Unit 3: Use the Copy Data Activity

The Core Function

The primary purpose of this activity is to ingest data from an external source and land it in **OneLake** (typically into a Lakehouse file or table). It is designed for "straight" moves where the data doesn't need to be changed during the transfer.

- **Standalone Use:** Moving a single file from a cloud provider into your Lakehouse.
- **Orchestrated Use:** Part of a larger chain. For example:
 1. **Delete Data activity:** Clears old records.
 2. **Copy Data activity:** Brings in the new batch of raw data.
 3. **Notebook activity:** Runs Spark code to transform that raw data into a structured table.

Key Features

- **Copy Data Tool:** A guided, graphical wizard that simplifies the process of connecting to a source and choosing a destination.
- **Wide Compatibility:** It supports a vast range of connections, including external SQL databases, cloud storage, and internal Fabric items like the [Data Warehouse or SQL Database](#).
- **Configuration Pane:** After placement on the canvas, you can use the settings pane to map columns, set retry logic, and define performance settings.

Strategic Choice: Copy Data vs. Dataflow

The unit provides a clear rule for when to choose this activity over others:

Use Copy Data If...	Use Dataflow (Gen2) If...
You want to move data "as-is" with no changes.	You need to transform, clean, or filter data <i>during</i> ingestion.
You plan to transform the data later (e.g., using a Notebook).	You need to merge data from multiple different sources into one.

You need a high-performance, simple move.	You prefer a Power Query-style interface for visual editing.
---	--

Summary

With Microsoft Fabric, you can create pipelines that encapsulate complex data ingestion and transformation processes. Pipelines provide an effective way to orchestrate data processing tasks that can be run on-demand or at scheduled intervals.

Use Apache Spark in Microsoft Fabric

Unit 1: Introduction to Apache Spark in Microsoft Fabric

Spark is defined as an open-source, parallel processing framework designed specifically for large-scale data analytics.

Key Takeaways:

- **Versatility:** Spark is a staple in "big data" scenarios and is implemented across various platforms, including [Azure HDInsight](#), [Azure Synapse Analytics](#), and [Microsoft Fabric](#).
- **Core Functionality:** You can use Spark to **ingest, process, and analyze** data directly within a Lakehouse environment.
- **The Fabric Advantage:** While Spark code is generally standardized, Microsoft Fabric offers an integrated environment that simplifies the process of combining Spark-based tasks with other data services.

Unit 2: Prepare to Use Apache Spark

This unit focuses on the infrastructure and configuration required to run **Apache Spark** within Microsoft Fabric, highlighting its "divide and conquer" approach to large-scale data processing.

1. Spark Pools

A [Spark pool](#) is the cluster of compute nodes that execute your data tasks. It consists of two main components:

- **Head Node:** Runs the **driver** program to coordinate distributed processes.
- **Worker Nodes:** Run **executor** processes to perform the actual data processing.

Microsoft Fabric provides a **starter pool** for quick setup, but you can create **custom pools** to adjust node family types, autoscale limits, and dynamic allocation.

2. Runtimes and Environments

- **Runtimes:** Determine the versions of Apache Spark, Delta Lake, and Python installed.
- **Environments:** Allow you to create custom configurations. You can install specific public libraries (from PyPI) or upload custom packages to ensure all nodes have the necessary tools for your specific workload.

3. Key Optimization Features

To improve performance and efficiency, Fabric offers several advanced settings:

Feature	Description
Native Execution Engine	A vectorized engine that speeds up queries on Parquet or Delta files.
High Concurrency Mode	Shares a single Spark session across multiple users or notebooks to save resources.
Automatic MLFlow Logging	Automatically tracks machine learning experiments without extra code.

Unit 4: Work with Data in a Spark Dataframe

This unit explores how to use the [Dataframe](#)—the most common structure for working with data in Spark—to load, transform, and save data within a Lakehouse.

1. Loading Data

You can ingest data from files (like CSVs) into a [dataframe](#) using two primary methods:

- **Inferring a Schema:** Spark automatically guesses column names and data types (e.g., `spark.read.load(..., header=True)`).
- **Explicit Schema:** You define the structure manually using `StructType`. This is **faster** and more reliable for complex data.

2. Manipulating Dataframes

Spark provides a "Dataframe API" that allows you to chain methods together for transformations:

- `select()`: Retrieves specific columns (similar to `SELECT` in SQL).
- `where()`: Filters rows based on conditions (similar to `WHERE` in SQL).
- `groupBy()` & `count()`: Aggregates data to find summaries.

3. Saving & Optimizing Results

After processing, you can write data back to the Lakehouse:

- **Parquet Format:** The preferred format for analytics due to its efficiency.
 - **Partitioning:** Using `partitionBy()`, you can split data into subfolders based on a specific column (e.g., `Category`). This drastically improves performance for future queries by reducing disk I/O.
-

Key Comparison for the SQL User

If you're thinking about your Stored Procedures, here's the translation:

- `df.where()` is your `WHERE` clause.
- `df.write.mode("overwrite")` is your `TRUNCATE TABLE` followed by an `INSERT`.
- **Partitioning** is similar to **Partitioned Tables** in SQL Server, helping the engine skip irrelevant data.

Unit 5: Work with Data using Spark SQL

This unit focuses on using [Spark SQL](#) to bridge the gap between procedural code (Python/Scala) and relational querying. It explains how to use the **Spark catalog** as a metastore for your data objects.

1. The Spark Catalog

The catalog acts as a library for your relational data. You can make data from a Dataframe available in the catalog in two ways:

- **Temporary Views:** Use `df.createOrReplaceTempView("view_name")`. These are session-based and disappear when the notebook stops. This is like a **Common Table Expression (CTE)** or a local temp table in SQL.
- **Tables:** Use `df.write.saveAsTable("table_name")`. These are persistent. In Microsoft Fabric, these are typically stored in the **Delta** format, which supports transactions and versioning.

2. Querying with the Spark SQL API

You can run SQL queries directly within your Python or Scala code using the `spark.sql()` method.

- **Example:** `df = spark.sql("SELECT * FROM products WHERE Price > 100")`
- This returns a new Dataframe, allowing you to switch back and forth between SQL logic and Python logic seamlessly.

3. The `%%sql` Magic Command

For those who prefer a pure SQL experience (similar to an SSMS query window), you can use the `%%sql` magic at the start of a notebook cell.

- This allows you to write standard SQL syntax to query tables and views.

- The results are automatically displayed as a formatted table in the notebook.

Comparison for the SQL Professional

SQL Server Concept	Spark SQL Equivalent
Local Temp Table (#table)	Temporary View
Permanent Table	Managed Table (Delta format)
Linked Server / External Table	External Table (Metadata in catalog, files in "Files" folder)
SSMS Query Tab	<code>%%sql</code> cell in a Notebook

Unit 6: Visualize Data in a Spark Notebook

This unit explains how to transform raw data into visual insights directly within a [Spark notebook](#). Visualization is often the final step in your "Stored Procedure" replacement workflow, allowing you to present results rather than just storing them.

1. Built-in Notebook Charts

When you run a query or display a dataframe, Microsoft Fabric provides an immediate, "no-code" way to see your data:

- **Instant Toggle:** You can switch the view from a **Table** to a **Chart** with a single click.
- **Customization:** You can use the built-in chart properties to change the chart type (Bar, Line, Pie, etc.) and configure the axes without writing extra code.

2. Visualization via Code (Graphics Packages)

For more complex or highly customized reports that the built-in tools can't handle, you can use industry-standard Python libraries.

- **Matplotlib:** The foundation for most Python plotting. It gives you full control over the figure size, colors, labels, and grid lines.

- **Pandas Integration:** Since most graphics libraries expect data in a [Pandas dataframe](#) format, you often use the `.toPandas()` method to convert your Spark results before plotting.
- **Seaborn:** An alternative library mentioned for creating more "polished" or statistically complex visualizations.

Key Comparison

Feature	Parquet Files	Delta Tables
User Experience	Like browsing a folder in Windows Explorer.	Like browsing a Database in SSMS.
Complexity	You must manage file paths and schemas.	The system manages metadata for you.
Tool Support	Great for Spark/Python developers.	Great for SQL Analysts and Power BI.
Version Control	Hard to track changes to data.	Supports Time Travel (viewing older versions of the data).

Real-time data in an Eventhouse

Unit 1: Introduction to KQL Databases in Microsoft Fabric

An [Eventhouse](#) is a specialized data store container designed to manage large volumes of continuous data streams using KQL databases.

Key Takeaways:

- **Real-Time Optimization:** Unlike traditional databases, KQL databases are built specifically for **time-series data**—information that is indexed by when it happened and is rarely changed once recorded (**immutable**).
- **The "Conveyor Belt" Model:** Data follows an **append-only pattern**. It flows in and is automatically partitioned by ingestion time, allowing the system to find recent data instantly without scanning the entire database.
- **Versatile Toolset:** Once data is ingested, you can query it using **Kusto Query Language (KQL)** or T-SQL, visualize it through **Real-Time Dashboards**, or trigger automated responses using **Fabric Activator**.
- **The Fabric Advantage:** Microsoft Fabric integrates these databases so you can seamlessly move from raw data ingestion to advanced analytics using materialized views and stored functions in one unified environment.

This unit focuses on the practical setup and querying capabilities of **Eventhouses** and **KQL databases** within Microsoft Fabric.

Unit 2: Get started with an Eventhouse

An [Eventhouse](#) is the primary workspace for managing real-time data, automatically providing a default KQL database for storing tables, stored procedures, and materialized views.

Key Takeaways:

- **Flexible Data Ingestion:** You can bring data into a [KQL database](#) from nearly anywhere—including local files, cloud storage (Azure S3, OneLake), and real-time streams like Kafka or Azure Event Hubs.
- **Zero-Copy Integration:** Through **database shortcuts**, you can query external databases without moving data. Additionally, **OneLake availability** ensures your KQL data is instantly accessible to other Fabric services like Power BI and Lakehouses.
- **Pipeline Query Logic:** KQL uses a **"funnel" approach** where data flows through pipes (`|`). Each operator (like `where`, `project`, or `summarize`) filters or transforms the data before passing it to the next step.
- **Advanced Analysis Tools:** The [KQL queryset](#) provides a dedicated environment to save queries, create visualizations, and even use **Copilot** to generate KQL code from natural language questions.

Unit 3: Use KQL effectively

[Query optimization](#) in KQL is centered on one main principle: the less data your query scans, the faster it runs and the fewer resources it consumes.

Key Takeaways:

- **Filter Early (The Funnel Principle):** Always apply filters—especially **time-based filters**—at the very beginning of your query. By using **where** statements early, you leverage KQL indexes to eliminate millions of unnecessary rows before any processing happens.
- **Column Reduction:** Use the **project** operator to select only the specific columns you need. Reducing "wide" tables down to a few essential columns significantly lowers the memory and processing overhead.
- **Smart Joins:** When joining two tables, always place the **smaller table first** (on the left side of the **join** operator). KQL loads the first table into memory to match against the second; using the smaller table first makes this process much more efficient.
- **Aggregated Exploration:** When testing queries or exploring large datasets, use the **limit** operator after aggregations to prevent the system from trying to display millions of result rows at once.

Unit 4: Materialized views and stored functions

[Materialized views](#) and [stored functions](#) are the primary tools used to handle complex aggregations and code reusability without sacrificing speed.

Key Takeaways:

- **Precomputed Performance:** [Materialized views](#) solve the "big data" lag by storing precomputed aggregation results. Instead of calculating a sum or average across billions of rows every time a dashboard refreshes, the system only processes new "delta" data and merges it with the existing results.
- **Freshness Guarantee:** Even though views are precomputed in the background, queries always return the most current data. The system automatically combines the **materialized part** (old data) with the **delta** (newly arrived data) at query time.
- **Code Encapsulation:** [Stored functions](#) allow you to save complex query logic as a reusable alias. This ensures that different team members apply the same filtering or transformation logic consistently across the organization.
- **Parameterized Queries:** Functions can accept parameters (like a specific ID or threshold). This allows you to treat a complex KQL query like a simple table call, such as `GetTripsByPassengerCount(3)`, making the code cleaner and more accessible for non-experts.

Implement a Lakehouse

Unit 1: Introduction to end-to-end analytics using Microsoft Fabric

[Microsoft Fabric](#) is an all-in-one analytics solution that consolidates disconnected data tools into a single, integrated environment built on a unified data lake.

Key Takeaways:

- **Solving Fragmentation:** Traditionally, organizations struggle with "data silos" where teams use separate, disconnected tools for ingestion, governance, and analysis. Fabric eliminates this complexity by providing one place for all data professionals to collaborate.
- **Unified Foundation (OneLake):** Every workload in Fabric sits on top of **OneLake**. Think of it as the "OneDrive for data"—a single, logical lake where data is stored once but can be used by various engines without duplication.
- **AI-Ready Infrastructure:** Because data is governed and organized in one spot, it is immediately ready for AI workloads. This includes [Copilots](#), machine learning models, and intelligent agents that require high-quality, centralized data to function effectively.
- **The Fabric Advantage:** The platform simplifies the entire lifecycle—from raw data ingestion to business-ready reports—ensuring that the same data powering a Power BI dashboard is the same data fueling an organization's AI initiatives.

Introduction to end-to-end analytics

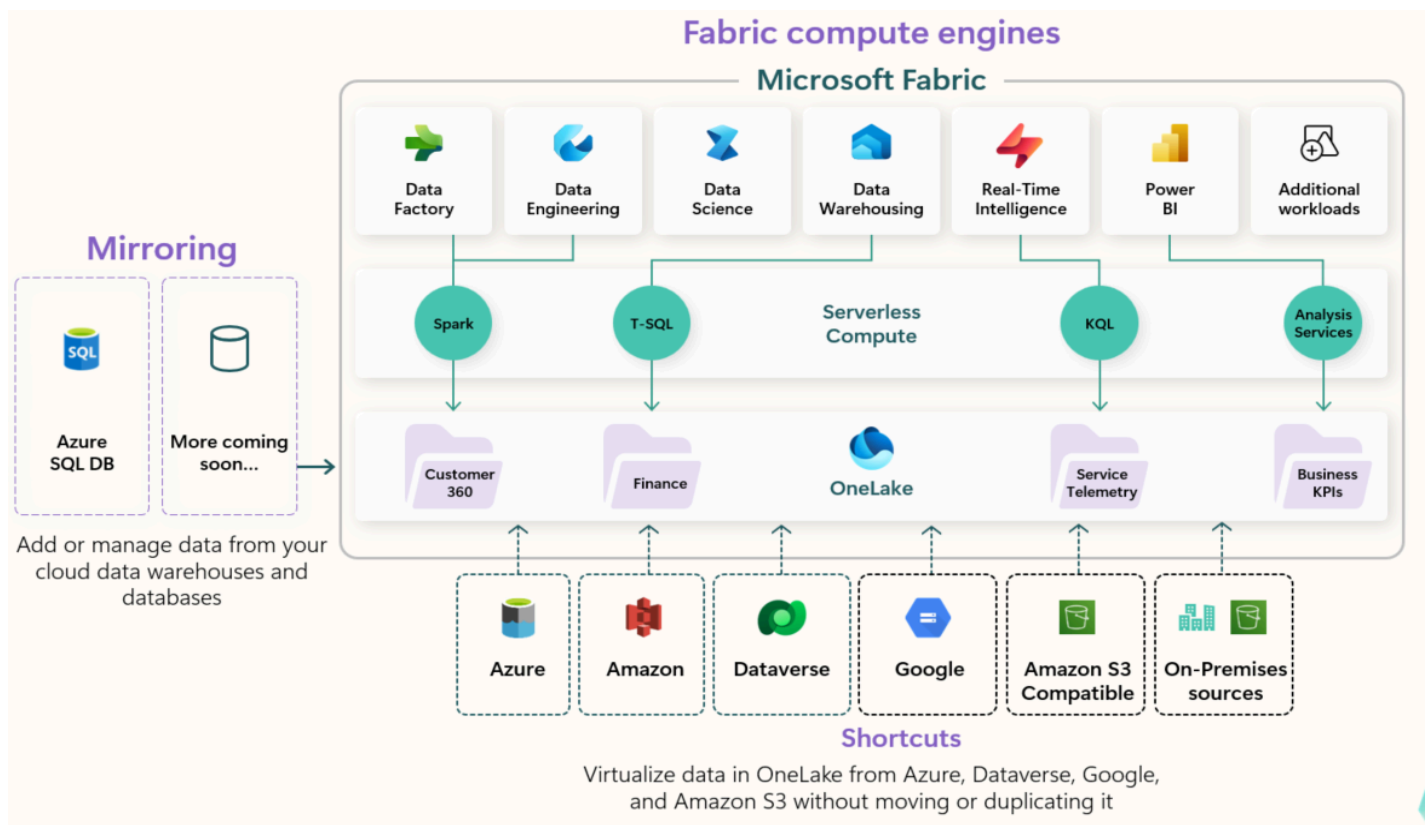
Unit 1: Introduction to end-to-end analytics using Microsoft Fabric

[Microsoft Fabric](#) is an all-in-one analytics solution that consolidates disconnected data tools into a single, integrated environment built on a unified data lake.

Key Takeaways:

- **Solving Fragmentation:** Traditionally, organizations struggle with "data silos" where teams use separate, disconnected tools for ingestion, governance, and analysis. Fabric eliminates this complexity by providing one place for all data professionals to collaborate.
- **Unified Foundation (OneLake):** Every workload in Fabric sits on top of **OneLake**. Think of it as the "OneDrive for data"—a single, logical lake where data is stored once but can be used by various engines without duplication.
- **AI-Ready Infrastructure:** Because data is governed and organized in one spot, it is immediately ready for AI workloads. This includes [Copilots](#), machine learning models, and intelligent agents that require high-quality, centralized data to function effectively.
- **The Fabric Advantage:** The platform simplifies the entire lifecycle—from raw data ingestion to business-ready reports—ensuring that the same data powering a Power BI dashboard is the same data fueling an organization's AI initiatives.

Unit 2: Explore end-to-end analytics with Microsoft Fabric



The core of the platform is [OneLake](#), a centralized "Data Lake" that allows all analytics engines to access the same data without duplication or movement.

Key Takeaways:

- **OneLake (The "OneDrive" for Data):** All data is stored in a single logical lake using open formats like **Delta-Parquet**. This ensures that whether you are using Spark, SQL, or Power BI, you are always looking at the same source of truth.
- **Shortcuts:** This feature allows you to "link" to data stored in other locations (like Amazon S3 or Azure Data Lake) without actually copying it. It keeps Fabric in sync with external sources automatically.
- **Unified Workspaces:** Assets are organized into [Workspaces](#), which serve as containers for team collaboration. They provide strict access control, version control through Git integration, and managed compute resources.
- **Centralized Governance:** Administration is handled through a single **Admin portal**. Tools like the **OneLake catalog** provide sensitivity labels and metadata tracking to ensure data remains secure and compliant across the entire organization.
- **AI Integration:** Because data is governed and stored in an open format within OneLake, it is natively accessible to **Copilot** and other AI agents without the need for additional data preparation pipelines.

Unit 3: Data teams and Microsoft Fabric

[Microsoft Fabric](#) removes data silos, allowing data engineers, scientists, and analysts to work together on the same foundation without duplicating efforts or moving data.

Key Takeaways:

- **Overcoming Traditional Barriers:** In standard workflows, teams often struggle with misinterpretation and delays due to fragmented tools. Fabric unifies these workflows into a single **SaaS platform**, ensuring everyone uses the same source of truth in [OneLake](#).
- **Role-Specific Integration:**
 - **Data Engineers:** Build the foundation by ingesting and transforming data directly into Lakehouses using Pipelines and Notebooks.
 - **Data Scientists:** Use integrated Spark notebooks and Python to build machine learning models that ground AI agents and Copilots.
 - **Data Analysts:** Connect directly to OneLake using **Direct Lake mode**, eliminating the need for time-consuming downstream transformations before creating Power BI reports.
- **The Analytics Engineer:** This role acts as the bridge, curating data within Lakehouses to ensure quality and building **semantic models** that make data accessible for self-service.
- **Empowering Low-Code Users:** Citizen developers can use the **OneLake catalog** and natural language queries via [Copilot](#) to find and analyze data without needing deep technical expertise.

- **AI Foundation:** Every role contributes to "AI-readiness." Clean data from engineers and business context from analytics engineers provide the essential grounding for accurate AI-driven insights.

Unit 4: Enable and use Microsoft Fabric

Before exploring Fabric's capabilities, an administrator must [enable it in the tenant settings](#). Once active, users can create collaborative environments called **workspaces** to house their data assets.

Key Takeaways:

- **Administrative Control:** Fabric is enabled through the **Admin portal**. Access can be granted to the entire organization or limited to specific security groups. Those without an enterprise setup can sign up for a [free Fabric trial](#).
- **The Workspace Engine:** Workspaces are the primary units of collaboration. They support **Git integration** for version control and provide **lineage views**, which visually map how data flows from source to final report.
- **OneLake Catalog:** This serves as the "search engine" for your data. Users can use the [OneLake catalog](#) to discover shared datasets, filtering by domain, keyword, or item type to find the right information quickly.
- **Fabric IQ & Data Agents (Preview):** A new layer of intelligence that uses **ontologies** to define business concepts. This allows **Data Agents** to understand "business language" (like "Revenue") rather than just raw table schemas when answering natural language questions.
- **Copilot Integration:** [Microsoft Copilot](#) is built into every workload. It assists with **code generation** (SQL, KQL, Python), data transformation guidance, and even the automatic creation of Power BI reports and summaries.

Data Engineering: Create lakehouses and operationalize workflows to build, transform, and share your data estate.

Data Factory: Ingest, transform, and orchestrate data.

Data Warehouse: Combine multiple sources in a traditional warehouse for analytics.

Real-Time Intelligence: Process, monitor, and analyze streaming data.

Industry Solutions: Use out-of-the-box industry data solutions.

Data Science: Detect trends, identify outliers, and predict values using machine learning.

Databases: Create and manage databases with tools to insert, query, and extract data.

IQ (preview): Unify data across OneLake and organize it according to the language of your business using **ontologies, graphs, and semantic models**.

Power BI: Create reports and dashboards to make data-driven decisions.

Get started with lakehouses

Unit 1: Introduction to Lakehouses in Microsoft Fabric

A [Lakehouse](#) is a unified architecture that combines the flexible, low-cost storage of a **data lake** with the structured querying and performance of a **data warehouse**.

Key Takeaways:

- **The Hybrid Approach:** Lakehouses are designed to handle both **structured** transactional data and **unstructured/semi-structured** data (like IoT logs or social media feeds) in a single location.
- **Unified Foundation (OneLake):** Built on the **OneLake** storage layer, a Lakehouse ensures that all data is stored once but remains accessible to multiple compute engines.
- **Dual Compute Engines:** It leverages **Apache Spark** for large-scale data processing and transformation, and **SQL compute engines** for high-performance querying and reporting.
- **Eliminating Silos:** By providing a single platform for all data types, the Lakehouse removes the need for complex integration efforts between separate storage systems and analytical tools.
- **Downstream Readiness:** Data organized within a [Lakehouse](#) is immediately available for advanced business intelligence, machine learning, and AI-powered experiences across Fabric.

Unit 2: Describe lakehouse features and capabilities

A [Lakehouse](#) bridges the gap between data lakes and warehouses by bringing structured, high-performance querying directly to your raw data.

Key Takeaways:

- **Dual-Folder Design:** The lakehouse organizes data into two distinct areas:
 - **Tables Folder:** Contains [Delta Lake tables](#) that support SQL queries, ACID transactions, and schema enforcement for reliable analytics.
 - **Files Folder:** A landing zone for raw data in any format (CSV, JSON, images). This provides the flexibility to stage and explore data before transforming it into structured tables.
- **Delta Lake Advantages:** At its core, the lakehouse uses the **Delta format**, which enables "Time Travel" (querying previous versions of data), efficient updates/deletes, and ensures data consistency even during simultaneous read/write operations.
- **Layered Security:** Fabric provides granular access control, including **workspace roles** for collaborators and **row-level/column-level security** through the SQL analytics endpoint to restrict sensitive data.
- **Foundation for AI:** By structuring data with clear schemas and naming conventions, the lakehouse becomes "AI-ready." This allows [Fabric IQ data agents](#) and Copilot to accurately translate natural language questions into business insights.

- **OneLake Integration:** Because it sits on **OneLake**, any table created in the lakehouse is instantly available for reporting in Power BI or processing in Spark without needing to move or copy the data.

Unit 3: Ingest and transform data in a lakehouse

The [lakehouse](#) operates in two modes: **Lakehouse explorer** for managing files and tables, and a **SQL analytics endpoint** for read-only T-SQL querying.

Key Takeaways:

- **Logical Organization:** Tables are stored in the **Tables folder** as Delta tables (structured), while raw data lives in the **Files folder** (unstructured). You can use **schemas** (like **dbo**, **sales**, or **hr**) to group tables logically and manage permissions.
- **Flexible Ingestion:** You can bring data in through multiple paths:
 - **Manual:** Direct [uploads](#) or "Load to Table" for a no-code experience.
 - **Automated: Data Factory pipelines** for external sources or **Dataflows Gen2** for a Power Query experience.
 - **Programmatic: Notebooks** using Spark (PySpark, Scala, or SQL).
- **Shortcuts (Zero-Copy):** [Shortcuts](#) allow you to reference data in other cloud storage (S3, ADLS Gen2) or other Fabric workspaces. The data appears as if it's local to your lakehouse, but it stays in its original source—saving you storage costs and sync time.
- **Transformation Tools:** Raw data in the "Files" area can be cleaned and moved to "Tables" using **Notebooks** (favored by engineers) or **Dataflows Gen2** (favored by analysts). **Pipelines** orchestrate these tasks to run in sequence or parallel.
- **AI Integration:** A clean schema organization is critical because [Fabric IQ data agents](#) use it to translate natural language questions into accurate SQL queries.

Unit 4: Query and analyze lakehouse data

Once data is ingested, you can choose between SQL-based exploration, code-based data science, or visual business intelligence without moving the data.

Key Takeaways:

- **The SQL Analytics Endpoint:** Every lakehouse automatically includes a [read-only SQL endpoint](#). This allows you to use standard T-SQL to query tables, create views, and apply row-level security—making it feel exactly like a traditional SQL database.
- **Spark Notebooks:** For more complex logic, you can use [Spark notebooks](#). You have the choice of **Spark SQL** for familiar queries or **PySpark** (Python) for advanced data manipulation and machine learning.
- **Direct Lake Mode:** This is a "game-changer" for Power BI. Instead of importing data (which takes time and duplicates files), [Power BI](#) reads the Delta Parquet files directly from OneLake. This provides the speed of "Import" with the real-time freshness of "Live Connection."
- **Semantic Models:** You can build [semantic models](#) on top of your lakehouse tables to define relationships and business measures. This creates a curated layer that business users can easily navigate in Power BI.

- **AI Assistance:** [Copilot](#) is integrated into both the SQL editor and Notebooks, allowing you to generate complex queries or Python code from simple natural language prompts.

Work with Delta Lake tables

This introductory unit highlights how **Microsoft Fabric** leverages the [Delta Lake](#) table format to combine the best of both worlds: the power of relational databases and the flexibility of data lakes.

Unit 1 Key Takeaways

- **The Foundation:** Tables in a Fabric lakehouse are based on the **Linux Foundation Delta Lake** format, an open-source storage layer designed for **Apache Spark**.
- **Relational Features on Big Data:** Delta Lake enables "lakehouse" architecture by providing:
 - **ACID Transactions:** Ensures data reliability during concurrent operations.
 - **Schema Enforcement:** Maintains data quality by preventing inconsistent structures.
 - **SQL Semantics:** Allows for familiar data manipulation within a Spark environment.
- **Simplified Management:** While Fabric manages much of the complexity for you, Unit 1 emphasizes that understanding the **metastore architecture** and specialized operations is key to building advanced analytics solutions.

Unit 2

of this module, [Understand Delta Lake](#), explains the technical structure and benefits of using Delta tables within a Microsoft Fabric lakehouse.

The Technical Structure

Delta tables are schema abstractions over data files. When you see the **Delta (Δ) icon** in the Fabric UI, the underlying storage consists of:

- **Parquet Files:** The actual data stored in a compressed, columnar format.
 - **_delta_log Folder:** A transaction log containing JSON files that track every change made to the table.
-

Key Benefits of Delta Lake

The unit outlines five primary advantages of using this format:

- **Relational Semantics:** Supports standard **CRUD** operations (Create, Read, Update, Delete). You can use **INSERT**, **UPDATE**, and **DELETE** just like in a traditional SQL database.
- **ACID Transactions:** Ensures data reliability through **Atomicity, Consistency, Isolation, and Durability**. This prevents data corruption during concurrent writes.
- **Time Travel:** Because every transaction is logged, you can access **data versioning** to query previous states of a table or "travel back in time" to recover data.

- **Unified Batch & Streaming:** Delta tables can act as both **sources** and **sinks**, allowing the same table to handle real-time data streams and historical batch processing simultaneously.
- **Interoperability:** Since the data is stored in the open **Parquet** format, it is highly compatible with various engines. In Fabric, you can query these tables using the **SQL analytics endpoint**.

Unit 3, Create delta tables,

focuses on the practical methods for defining and saving Delta tables within a Microsoft Fabric lakehouse using Apache Spark.

1. Managed vs. External Tables

The unit distinguishes between how Fabric handles table data and metadata:

- **Managed Tables:** Both the metadata and the data files are managed by the Fabric lakehouse. If you delete the table from the metastore, the underlying Parquet files are also deleted.
- **External Tables:** The metadata is in the metastore, but the data is stored in a separate location (like the **Files** folder or an external URL). Deleting the table definition **does not** delete the actual data files.

2. Methods for Creating Tables

There are several ways to define a Delta table depending on your workflow:

Method	Best For...
From a Dataframe	Quickly saving processed data using <code>df.write.format("delta").saveAsTable("name")</code> .
DeltaTableBuilder API	Programmatically defining a table schema (columns and types) from scratch using Python.
Spark SQL	Using familiar CREATE TABLE syntax, which is ideal for those coming from a relational database background.

3. Saving Data in Delta Format

You can also save data in the Delta format without immediately defining a table in the metastore. This is done by saving a dataframe to a specific path:

- **Overwrite Mode:** Replaces existing data in the folder.
- **Append Mode:** Adds new rows to the existing data.
- **Automatic Discovery:** A unique Fabric "Tip" mentions that if you save Delta files directly to the **Tables** folder in the lakehouse, Fabric will **automatically discover** them and create the metadata for you.

Unit 4 - Use Delta Tables in Apache Spark

The [Optimize delta tables](#) module outlines how to manage and improve the performance of Delta tables within **Microsoft Fabric**. It specifically focuses on overcoming the "small file problem" caused by Spark's parallel processing and the immutable nature of Parquet files.

Here are the key features and techniques covered:

1. Performance Optimization Tools

- **OptimizeWrite:** A background feature that reduces the number of small files during the write process. It is enabled by default in Fabric but can be toggled via Spark configuration.
 - **OPTIMIZE Command:** A manual maintenance task that consolidates existing small Parquet files into larger, more efficient ones.
 - **V-Order:** A Fabric-specific write optimization that sorts and compresses data to enable "lightning-fast" reads. While it adds about 15% overhead to write times, it significantly boosts read speeds for Power BI and SQL engines using **Verti-Scan** technology.
-

2. Storage Management

- **VACUUM:** This command permanently deletes old Parquet data files that are no longer referenced in the transaction log and fall outside the retention period (default is 7 days).
 - **Note:** Running VACUUM prevents "time travel" to versions older than the specified threshold.
-

3. Data Layout Strategy

- **Partitioning:** Organizing data into subfolders (e.g., by **Year** or **Category**) to enable **data skipping**.
- **Best Practices:**
 - * **Use it for:** Very large datasets with clear, low-cardinality filtering columns.
 - **Avoid it for:** Small datasets or columns with high cardinality (too many unique values), as this can actually worsen performance by creating too many small files.

Summary Table: When to use what?

Feature	Primary Benefit	When to Use
OptimizeWrite	Prevents small files	Always (enabled by default).
OPTIMIZE	Consolidates existing files	After large data loads or when query performance drops.
V-Order	Faster reads (Power BI/SQL)	For tables that are read frequently by reporting tools.
VACUUM	Saves storage costs	Regularly, to clean up obsolete data files.
Partitioning	Speeds up filtered queries	Only on very large tables with logical filter columns.

Unit 5 - Use delta tables with streaming data

1. Using Spark SQL

The most common approach for working with Delta tables is through SQL commands.

- **Integration:** You can embed SQL statements within PySpark or Scala using `spark.sql()`.
- **Notebook Magic:** Use the `%%sql` magic command in Fabric notebooks to run native SQL for tasks like `INSERT` or `UPDATE`.

2. Using the Delta Lake API

When working directly with file locations rather than catalog tables, the **Delta Lake API** is often more efficient.

- **Functionality:** You can create a `DeltaTable` object using a file path.
- **Programmatic Updates:** It allows for structured updates and deletes, such as applying a conditional price reduction across a specific category.

3. Time Travel and Versioning

Delta Lake automatically logs every transaction, enabling "Time Travel" to view or restore previous states of the data.

- **History:** Use the `DESCRIBE HISTORY` command to see a log of versions, timestamps, and operations.
- **Retrieving Past Data:** You can load a specific version of a table into a DataFrame by using:
 - `versionAsOf`: To access a specific version number (e.g., Version 0).
 - `timestampAsOf`: To access the data as it existed at a specific date and time.

Unit 6 - Work with Delta Lake tables

This unit focuses on using **Delta tables** as both sources and sinks within a **Spark Structured Streaming** architecture in Microsoft Fabric.

Core Concepts

Spark Structured Streaming treats real-time data as a "boundless dataframe," allowing you to process live data using the same APIs as static batch data.

- **Streaming Source:** Reading data from a Delta table as it is appended.
- **Streaming Sink:** Writing processed data streams into a Delta table for persistence and further analysis.

Key Technical Workflows

1. Using Delta as a Source

When you use a Delta table as a source, Spark monitors the table's transaction log to identify new data.

- **Code Example:**
- Python

```
stream_df = spark.readStream.format("delta").table("orders_in")
```

-
-

- **Important Note:** By default, Delta sources expect only **append** operations. If data is deleted or updated in the source, you must use options like `ignoreChanges` or `ignoreDeletes` to prevent errors.

2. Transforming the Stream

Once data is in a streaming DataFrame, you can apply standard Spark functions to filter or modify it in real time (e.g., filtering `NULL` values or calculating new columns like `Total` price).

3. Using Delta as a Sink

To save the streaming data, you define a Delta table as the output.

- **Checkpoints:** You must specify a [checkpointLocation](#). This stores the state of the stream, allowing it to resume exactly where it left off if a failure occurs.
- **Code Example:**

Python

```
deltastream = transformed_df.writeStream.format("delta") \
    .option("checkpointLocation", checkpointpath) \
    .start(output_table_path)
```

Summary Table

Feature	Description
API	Spark Structured Streaming
Latency	Near real-time processing.
Reliability	Managed via Checkpoints to track progress and recover from failures.
Lifecycle	Streams must be explicitly stopped (using .stop()) to avoid ongoing compute costs.

Fabric lakehouse using medallion
architecture

Unit 1 Organize a Fabric lakehouse using medallion architecture design

he [Introduction - Training | Microsoft Learn](#) page provides an overview of the **medallion architecture** within the context of a **Microsoft Fabric lakehouse**.

Here is a summary of the key points:

Core Concept

The medallion architecture is an industry-standard design pattern used to organize, refine, and curate data in a lakehouse environment. It allows businesses to blend the flexibility of data lakes with the structured performance of data warehouses.

The Three Layers

The module introduces the logical flow of data through three distinct stages:

- **Bronze:** The raw data landing zone.
- **Silver:** Validated and enriched data.
- **Gold:** High-quality, business-ready data curated for reporting and analytics.

Unit 2 Describe medallion architecture

This unit from [Microsoft Learn](#) explains the **medallion architecture**, a data design pattern used to logically organize a lakehouse into layers that increase in data quality and refinement.

The Three Standard Layers

- **Bronze (Raw):** The initial landing zone. Data is stored in its original format (structured or unstructured) without any modifications.
 - **Silver (Validated):** Data is cleaned, merged, and deduplicated. It acts as a consistent, "source of truth" repository accessible by multiple teams.
 - **Gold (Enriched):** Data is further refined and aggregated to meet specific business needs. This layer is optimized for analytics, data science, and reporting.
-

Moving Data in Microsoft Fabric

The unit distinguishes between two critical processes for moving data through these layers:

1. **Data Transformation:** Altering the structure or content.
 - **Dataflows (Gen2):** Best for smaller models and simple logic.
 - **Notebooks:** Best for large-scale data and complex transformations.
2. **Data Orchestration:** Managing the "flow" and timing of tasks.

- **Pipelines:** The primary tool used to schedule and automate the movement of data from one layer to the next.

Key Takeaway

The medallion architecture is **flexible**. While the three-layer approach is standard, organizations can add custom layers (like "Platinum") to suit specific requirements. It is designed to work alongside existing data models, not replace them.

Implement Real-Time Intelligence

Get started with Real-Time Intelligence

Unit 1 Introduction to Real-Time Intelligence in Microsoft Fabric

Overview: This unit introduces the concept of Real-Time Intelligence within the Microsoft Fabric ecosystem, focusing on its application in high-stakes operational environments like logistics and delivery services.

1. The Business Problem: Batch vs. Real-Time

Traditional data processing often relies on **batch reporting**, where data is collected and processed in intervals (e.g., overnight). In a delivery company scenario, this leads to significant lag:

- **Delayed Awareness:** Issues like vehicle breakdowns or weather delays are only identified after the fact.
- **Information Gap:** Customers remain uninformed about delays in real-time.
- **Reactive Operations:** Decisions are made based on old data rather than current conditions.

2. The Solution: Microsoft Fabric Real-Time Intelligence

Real-Time Intelligence shifts the focus to **data in motion**. By analyzing streaming data as events occur, organizations can achieve:

- **Continuous Monitoring:** Tracking GPS trackers, warehouse scanners, and feedback systems instantly.
- **Instant Analysis:** Identifying route delays and estimating delivery windows based on live conditions.
- **Automated Workflows:** Setting up customer notifications and route optimization triggers automatically.

3. Key Learning Objectives

By the end of this module, you will understand how to:

Objective	Description
Component Integration	Understand how Microsoft Fabric components work together for real-time solutions.
Data Lifecycle	Learn to ingest, process, store, and query data as it arrives.

Visualization & Automation	Visualize "data in motion" and create automated responses to changing conditions.
---------------------------------------	---

Unit 2 [What is real-time data analytics?](#)

Overview: This unit defines real-time analytics as the practice of processing and acting on data as it is generated, typically within seconds or minutes. It highlights the shift from analyzing static historical data to managing "data in motion."

1. Core Concepts: Events and Streams

- **Events:** Individual digital records of specific actions or changes (e.g., a website click, a sensor reading, or a stock price change).
- **Streams:** A continuous, chronological sequence of events. Streams act as the delivery mechanism that moves data from the source to the processing engine.

2. Essential Components of a Solution

To move from raw data to actionable insights, a real-time solution requires:

- **Real-time Ingestion:** Collecting data from diverse sources (logs, APIs, sensors) simultaneously.
- **Stream Processing:** Transforming, filtering, and aggregating data while it is in transit.
- **Low-latency Storage:** Databases optimized for high-velocity writes and near-instant queries.
- **Interactive Dashboards:** Visualizations that update automatically as new data arrives.
- **Automated Decision Making:** Triggers and rules that initiate alerts or workflows based on live conditions.

3. Business Value

By implementing real-time analytics, organizations can:

- **Respond Immediately:** Act on emerging opportunities or threats without lag.
- **Optimize Operations:** Adjust resources dynamically based on current demand.
- **Enhance Experience:** Provide personalized, contextual interactions to customers.
- **Prevent Issues:** Detect anomalies and fix them before they escalate into critical failures.

4. Integration in Microsoft Fabric

Microsoft Fabric provides a unified platform for these tasks through:

- **Eventstreams:** For ingestion and transformation.
- **Eventhouses:** For optimized storage.
- **Real-Time Hub:** For data discovery.
- **Activator:** For automated alerts and actions.

Unit 3: Real-Time Intelligence in Microsoft Fabric

Overview: This unit explores the specific components within Microsoft Fabric that enable end-to-end processing of streaming data, from initial capture to automated response.

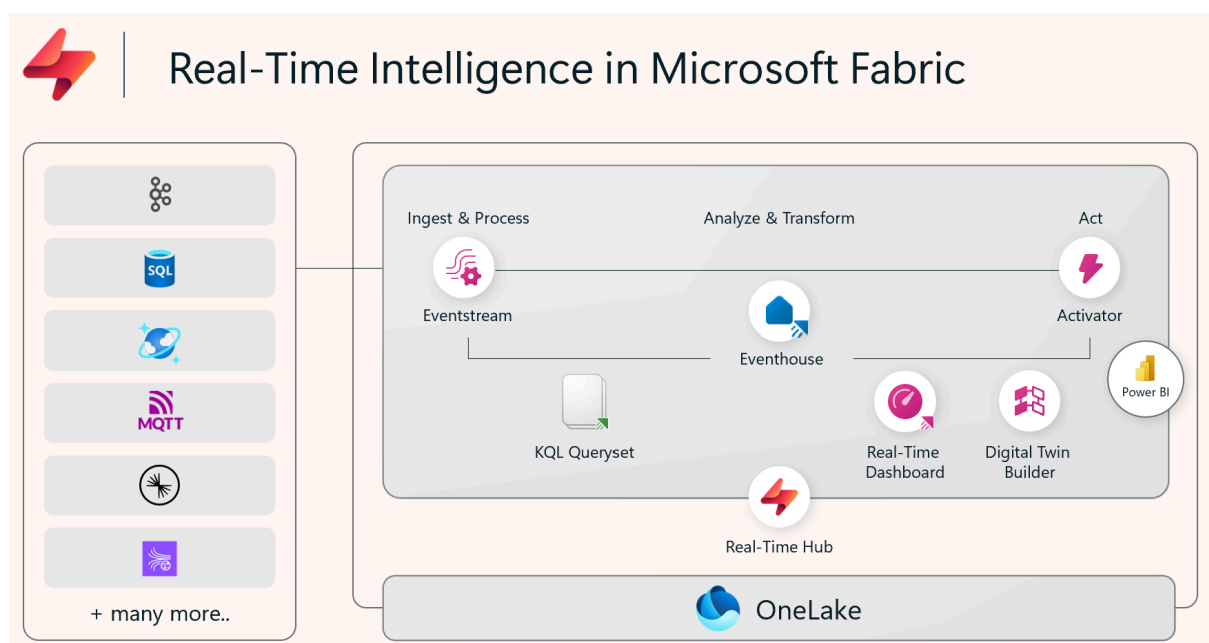
1. Key Use Cases

Real-Time Intelligence is designed for scenarios where "near real-time" insights are critical:

- **Delivery Tracking:** Alerting customers about package delays via live vehicle monitoring.
- **Equipment Monitoring:** Tracking sensor data (like temperature) to prevent mechanical failure.
- **Fraud Detection:** Instantly blocking suspicious transactions.
- **System Health:** Monitoring application errors and website load times to maintain service reliability.

2. Core Components of the Ecosystem

Microsoft Fabric uses a suite of integrated tools to manage the data lifecycle:



Component	Primary Function
Eventstreams	Captures, filters, enriches, and transforms data as it flows into the system.
Eventhouse	Stores high-velocity time-series data in KQL databases, integrated with OneLake.
KQL Queryset	A workspace to run and manage Kusto Query Language (and T-SQL) queries.
Real-Time Dashboard	Provides auto-refreshing visualizations to monitor live conditions and trends.
Activator	Triggers automated actions (notifications or workflows) when specific thresholds are met.

3. The Real-Time Hub

The [Real-Time Hub](#) acts as a central catalog for all "data-in-motion." It organizes data into four main categories:

- **Data Sources:** Connectors for Microsoft, database CDC feeds, and external cloud providers.
- **Azure Sources:** Direct configuration for IoT Hub, Service Bus, and Data Explorer.
- **Fabric Events:** Subscriptions to system events (e.g., job status changes or OneLake file actions).
- **Azure Events:** Subscriptions to Azure service events (e.g., Blob Storage changes).

Unit 4: [Ingest and transform real-time data](#)

Overview: This unit focuses on the two primary methods for bringing streaming data into Microsoft Fabric: using **Eventstreams** for active processing or **Direct Ingestion** for high-velocity storage.

1. Eventstreams (In-Transit Transformation)

Eventstreams act like a "water pipe system" where data is processed while it is moving.

- **Sources:** You can connect to [Microsoft sources](#) (Azure Event Hubs, IoT Hub), [Fabric/Azure events](#) (OneLake changes), and [External platforms](#) (Apache Kafka, MQTT).
- **Transformations:** Data can be cleaned or reshaped *before* storage using tools like SQL code, filters, aggregations, and joins.
- **Destinations:** Processed data is routed to a KQL database, Lakehouse, Activator, or custom endpoints.

2. Direct Ingestion (Post-Ingestion Transformation)

For scenarios requiring speed over immediate reshaping, data can be sent directly to a [KQL database](#) in an Eventhouse.

- **Sources:** Direct ingestion supports local files, Azure Storage, Amazon S3, and OneLake.
- **Update Policies:** Unlike Eventstreams, transformation happens *after* the data lands in the database. [Update policies](#) are automated triggers that run a query to transform newly arrived data and save it to a final destination table.

3. Comparison of Approaches

Feature	Eventstreams	Direct Ingestion
Transformation Timing	Before data is stored (in-flight).	After data lands (via Update Policies).
Primary Tool	Event Processor / SQL Editor.	KQL Queries / Connectors.
Best For	Complex routing and multi-destination flows.	High-volume raw data storage and rapid querying.

Unit 5: [Store and query real-time data](#)

Overview: This unit covers how to manage and analyze data once it lands in an **Eventhouse**. It highlights the use of **KQL Databases** for storage and the **Kusto Query Language (KQL)** for high-performance analysis.

1. Storage Components in an Eventhouse

Within an Eventhouse, you work with two primary structures:

- **KQL Databases:** These are real-time optimized data stores that host tables, stored functions, materialized views, and data streams.
- **KQL Querysets:** A dedicated space for writing and saving queries. These support both KQL and a subset of T-SQL.

2. Kusto Query Language (KQL)

KQL is the primary language used for querying large volumes of structured and semi-structured data. It is optimized for **time-series data** and uses a pipe-based (|) syntax.

Basic Syntax Example:

Code snippet

stock

| where ["time"] > ago(5m)

| summarize avgPrice = avg(todouble(bidPrice)) by symbol

- **Performance:** Data is indexed by ingestion time and partitioned automatically for speed.
- **Familiarity:** If you aren't ready for KQL, you can use **T-SQL** (e.g., `SELECT TOP 10 * FROM stock`) or leverage **Copilot** to generate queries using natural language.

3. Advanced Automation & Tools

To go beyond simple queries, you can use **Management Commands** to automate data processing:

- **Update Policies:** Automatically transform and move data into different tables as it arrives.
- **Materialized Views:** Pre-calculate summary results so complex queries run faster.
- **Stored Functions:** Save and reuse frequently used query logic.
- **Copilot for Real-Time Intelligence:** An AI assistant that helps translate your business questions into functional KQL code.

Unit 6: [Visualize real-time data](#)

Overview: This unit explains how to transform raw query results into actionable, visual insights using **Real-Time Dashboards** and **Power BI** within the Microsoft Fabric environment.

1. Real-Time Dashboards

[Real-Time Dashboards](#) are single-page interfaces designed to show live data at a glance.

- **Tiles:** Dashboards are made of "tiles." Each tile runs a specific **KQL query** to extract data from an Eventhouse.
- **Interactivity:** Once published, users can interact with tiles to drill deeper into data, apply filters, or change the visualization type on the fly.
- **Creation Methods:** You can start a dashboard directly from a workspace or promote a query from a **KQL queryset** into a dashboard tile.

2. Visualization via Power BI

For more traditional or complex reporting, Fabric allows you to build [Power BI reports](#) directly from your KQL database.

- **Seamless Integration:** Power BI can connect to the KQL database to provide high-level business analytics alongside streaming data.
- **Rich Reporting:** This is ideal for combining real-time metrics with historical business data for a comprehensive overview.

3. Key Benefits

- **Instant Visibility:** No need to manually refresh; visualizations update as the data changes.
- **Customization:** Edit tiles to show tables, charts, or graphs depending on what best represents the data.
- **Data Discovery:** Use the visual interface to aggregate and explore "data in motion" without writing additional code.

Use Eventstream

UNIT 1

This module introduces **Eventstream** in Microsoft Fabric, a feature within Real-Time Intelligence designed to manage data in motion without writing code.

Key Capabilities

- **Capture & Route:** Effortlessly collect real-time events and direct them to various destinations.
- **No-Code Transformation:** Use a drag-and-drop interface to process and transform data logic visually.
- **Versatile Connectivity:** Ingest [Azure IoT Hub](#) data from diverse sources including:
 - [Azure Event Hubs](#)
 -
 - [Apache Kafka](#)

Objective

The primary goal of the [Eventstream](#) module is to teach users how to build end-to-end pipelines that ingest, transform, and store real-time data for advanced analysis.

UNIT 2 Components of Eventstream

The **Eventstream** feature in Microsoft Fabric is a no-code, visual solution for building real-time data pipelines. It acts as a delivery mechanism that captures events as they happen and routes them to specific locations for analysis or action.

The system is built around three core components:

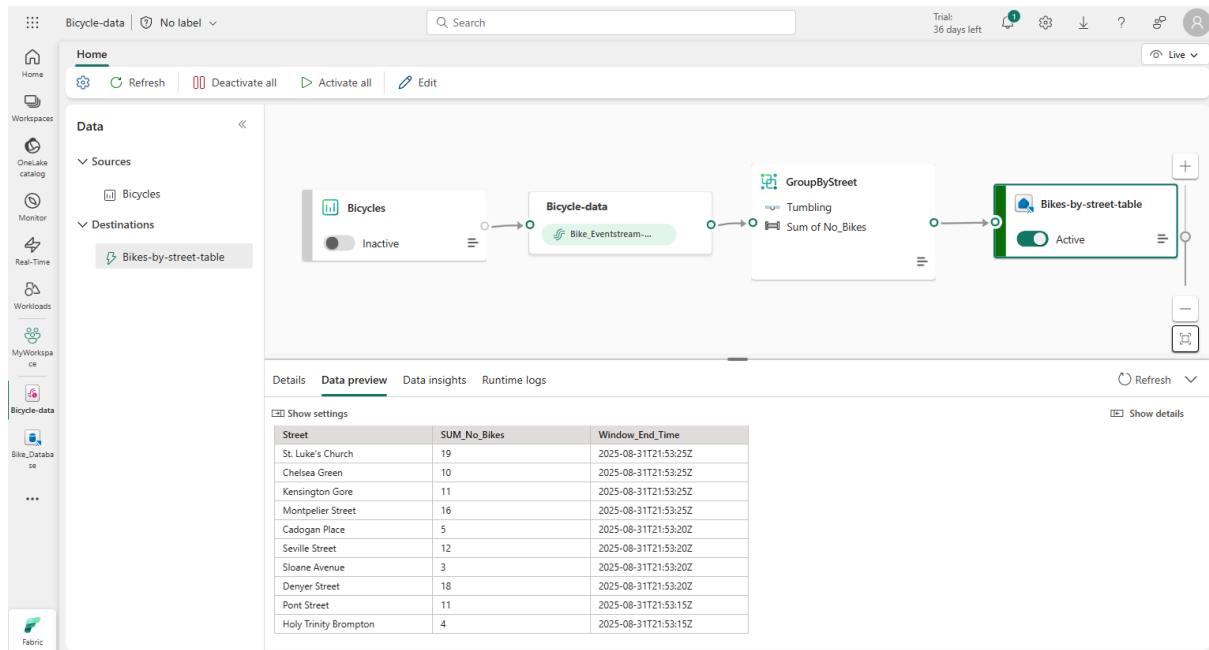
1. Sources

These are the origins of your data. You can ingest real-time events from both **Microsoft sources** and **non-Microsoft platforms**.

2. Transformations

Before the data reaches its destination, you can reshape it using a drag-and-drop canvas. Available operations include:

- **Filtering** and **Aggregating** data.
- **Grouping** (e.g., summing values by a specific category).
- Using **SQL code** for complex logic.
- **Joining** or expanding fields.



3. Destinations

This is where the processed data is sent for storage or further use. Common destinations include:

- [Eventhouse](#) or **Lakehouse** tables.
- **Fabric Activator** (to trigger automated actions based on the data).
- **Custom endpoints** or derived streams for additional processing.

Unit 3 Eventstream sources and destinations

This unit covers the configuration of **sources** and **destinations** within a Microsoft Fabric [Eventstream](#), highlighting how data flows from external origins to actionable endpoints.

1. Eventstream Sources

You can ingest real-time data from a variety of platforms using the eventstream canvas:

- **Microsoft Sources:** [Azure Event Hubs](#), [IoT Hubs](#), [Service Bus](#), and CDC feeds.
- **Fabric/Azure Events:** Changes in [OneLake](#) data stores or [Blob Storage](#) events.
- **External Sources:** Popular third-party platforms like [Apache Kafka](#), [Google Cloud Pub/Sub](#), and [MQTT](#).

2. Eventstream Destinations

Once data is ingested (and optionally transformed), it can be routed to multiple endpoints simultaneously:

- **Eventhouse**: For high-speed ingestion and analysis using KQL.
- **Lakehouse**: Converts events into Delta Lake format for storage in lakehouse tables.
- **Fabric Activator**: Triggers automated actions (like notifications or workflows) based on specific data patterns.
- **Derived Stream**: Acts as a "sub-stream" to enable **content-based routing**, allowing you to send different subsets of data to different destinations.
- **Custom Endpoint**: Routes data to external systems or custom applications outside of Fabric.

Unit 4 Eventstream transformations

This unit focuses on **Eventstream transformations**, which allow you to clean, enrich, and reshape raw streaming data before it reaches its final destination.

Why Transform Data?

Raw data is rarely "analysis-ready." Transformations help achieve:

- **Data Quality**: Filtering out noise or incomplete records.
- **Enrichment**: Adding calculated fields or renaming columns for clarity.
- **Standardization**: Ensuring consistent formats across different data sources.
- **Summarization**: Calculating averages or totals over specific time windows.

No-Code Transformation Tools

The eventstream canvas provides several drag-and-drop tiles to process data:

- **Filter**: Keeps only the events that meet specific conditions (e.g., **Temperature > 80°**).
- **Manage Fields**: Used to add, remove, rename, or change the data type of columns.
- **Aggregate**: Calculates values like **Sum**, **Min**, **Max**, or **Average** in real-time as events arrive.
- **Group By**: Performs aggregations over specific time intervals, such as **Tumbling windows** (fixed blocks) or **Sliding windows** (overlapping intervals).
- **Union & Join**: Combines multiple streams into one or matches data from two different streams based on a shared key.
- **Expand**: Breaks down arrays into individual rows.

Create Real-Time Dashboards

Unit 1 Introduction

The Core Purpose

The primary goal of Real-Time Dashboards is to monitor **constantly changing data** to enable immediate operational decisions. Unlike traditional reporting tools that provide historical snapshots, these dashboards are designed for high-frequency refreshing.

Key Technical Components

- **Data Source:** The dashboards connect to **KQL databases** located in **eventhouses**.
- **Visualizations:** They display data through visualizations that refresh automatically to reflect the most current information available.
- **Use Case Example:** The unit uses a bike-sharing company as an example, where an analyst needs to monitor live bike availability and dock capacity to redistribute bikes to high-demand areas effectively.

Comparison: Traditional vs. Real-Time

Feature	Traditional Reports	Real-Time Dashboards
Data Focus	Historical snapshots	Current, live data streams
Refresh Rate	Scheduled/Infrequent	High-frequency/Automatic
User Action	Long-term trend analysis	Immediate operational response

Unit 2 - Real-time dashboards

Unit 3 of the [Create Real-Time Dashboards with Microsoft Fabric](#) module focuses on **Organizing and Filtering Dashboard Data**.

Building on the basic tile creation from Unit 2, this section covers how to make your dashboards scalable and interactive. Here are the core highlights:

1. Base Queries

To make your dashboard more efficient and easier to maintain, you can use **Base Queries**.

- **Efficiency:** Instead of writing the same KQL code in every tile, you create a single "source" query.

- **Consistency:** Multiple tiles (e.g., a map and a bar chart) can reference this base query, ensuring they all use the exact same filtered dataset.
- **Variable usage:** Once defined, you reference it in individual tiles like a variable (e.g., `base_bike_data`).

2. Parameters

Parameters add **interactivity** for the end-user.

- **Dynamic Filtering:** You can add dropdowns or text boxes (like a "Neighborhood" selector) to the dashboard toolbar.
- **KQL Integration:** These parameters are injected into your Kusto queries using the `$parameter_name` syntax, allowing the visuals to update instantly based on user selection.

3. Multi-Page Layouts

For complex monitoring needs, you can organize tiles across **multiple pages**.

- This prevents "dashboard clutter" and allows you to group related metrics (e.g., one page for "Live Availability" and another for "Technical Health").

Key Summary Table

Feature	Primary Benefit
Base Queries	Simplifies maintenance; reduces redundant code across tiles.
Parameters	Enables end-users to slice and dice data without editing queries.
Pages	Organizes content for better readability and logical grouping.

Unit 3 Organize and filter dashboard data

This unit details how to [Organize and filter dashboard data](#) within Microsoft Fabric to create more effective and interactive Real-Time Dashboards.

Key Features for Dashboard Organization

- **Pages:** You can add multiple pages to a single dashboard to serve as containers for different sets of tiles. This is useful for grouping related content, such as separating data by source or subject area.
- **Parameters:** These add flexibility by allowing users to filter the data displayed in tiles (e.g., filtering by time period or a specific subset of data).
 - **Variables:** Parameters use a variable name with an underscore prefix (e.g., `_selected_neighbourhoods`).
 - **KQL Integration:** You reference these variables directly in your Kusto Query Language (KQL) code to make tiles dynamic based on user selection.
- **Auto Refresh:** This feature ensures the dashboard updates automatically without manual reloads.
 - **Editors** can set default and minimum refresh rates to manage system performance.
 - **Viewers** can adjust the rate during their specific session within the limits set by the editor.

Unit 3 Dashboard management and optimization

This unit, [Dashboard management and optimization](#), focuses on improving the efficiency and maintainability of your Microsoft Fabric dashboards using **Base Queries**.

The Power of Base Queries

Instead of writing a unique, heavy query for every single tile, you can create a single "source" query that handles the bulk of the data retrieval.

- **Efficiency:** It prevents duplicating the same logic across multiple tiles.
- **Performance:** It reduces the load on your KQL database by fetching a general set of records once and then letting individual tiles filter that specific subset.
- **Maintenance:** If you need to change the data source or a core filter, you only have to update the base query in one place rather than editing every tile.

How It Works

1. **Define the Base:** You create a query in the dashboard's "Base queries" menu and assign it a variable name (e.g., `_base_bike_data`).
2. **The Logic:** This query usually fetches a broad set of data, such as:
3. Code snippet

Bikes

```
| where ingestion_time() between (ago(30min) .. now())
| summarize latest_observation = arg_max(ingestion_time(), *) by Neighbourhood
```

4. **The Reference:** You then create individual tiles that start with that variable name instead of the raw table:
5. Code snippet

`_base_bike_data`

| project Neighbourhood, latest_observation, No_Bikes

Summary of Benefits

Feature	Benefit
Logic Reuse	Write once, use across many visuals.
Variable Mapping	Use simple names like <code>_base_data</code> to keep tile code clean.
Optimization	Faster dashboard loading by streamlining how data is pulled.

Use Activator in Microsoft Fabric

Unit 1 Use Activator in Microsoft Fabric

This unit introduces [Activator](#), a real-time event detection and rules engine within Microsoft Fabric designed to trigger automated actions based on live data patterns.

Core Functionality

Activator operates through a three-step process:

1. **Connect:** Link to real-time data sources (e.g., streaming sensor data).
2. **Monitor:** Create **rules** to evaluate specific conditions or thresholds.
3. **Act:** Execute **actions** when conditions are met, such as sending notifications or starting automated workflows.

Key Action Types

- **Notifications:** Email alerts or Microsoft Teams messages.
- **Workflows:** Triggering Power Automate flows.
- **Automation:** Running Microsoft Fabric notebooks to process data or adjust systems.

Real-World Applications

Activator is used across various industries where "time-to-response" is critical:

- **Manufacturing:** Alerting teams when equipment exceeds safe temperatures.
- **Logistics:** Monitoring cold-chain integrity for sensitive goods (like medicine).
- **Retail:** Automating inventory reorders when stock hits a critical low.
- **Finance:** Flagging unusual transaction patterns for immediate fraud review.

Unit 2 Configure Activator for your data

Unit 2 focuses on how to [configure Activator](#) to interpret real-time data by organizing it into meaningful business entities.

The Business Object Model

To monitor data effectively, Activator uses a hierarchy to bridge the gap between raw data streams and real-world entities:

- **Objects:** Represent individual real-world instances (e.g., a specific package, a delivery truck, or a customer).
- **Properties:** The specific attributes of that object that you want to track (e.g., temperature, current city, or hours in transit).

- **Events:** The raw incoming data points that update these properties.
-

Setting Up Data Monitoring

The primary way to configure these objects is through **Fabric Eventstreams**. The process involves:

1. Setting Activator as the **destination** for your eventstream.
2. Selecting a **Unique Identifier** (like **DeviceID** or **PackageID**) so Activator knows which events belong to which object.
3. Mapping data fields to **Properties** that will be evaluated against rules.

Alternative Alerting Approaches

While business objects are the standard for tracking specific entities, the [Activator engine](#) also supports:

- **Dashboard Alerts:** Created directly from Real-Time Dashboard visuals.
- **System Event Alerts:** Monitoring Fabric workspace activities or OneLake file operations.
- **Query Alerts:** Triggered from KQL Queryset results.

Unit 3 Create rules in Activator

Unit 3 of the [Use Activator in Microsoft Fabric](#) module explains how to build rules that detect specific conditions in your data. It breaks the configuration down into three primary sections within the **Definition** pane:

1. Choose What to Monitor

The **Monitor** section defines the data attribute you are watching. Because raw data can be "noisy" (e.g., a momentary temperature spike), Activator uses **Summarization** to clarify trends:

- **Operations:** You can choose *Average*, *Minimum/Maximum*, *Count*, or *Total*.
- **Window Size:** Determines how much historical data to include (e.g., the last 10 minutes).
- **Step Size:** Determines how often the rule is recalculated (e.g., every 5 minutes).

2. Define When to Act

The **Condition** section sets the specific threshold or pattern that triggers the rule:

- **Detection Types:** Includes *Threshold monitoring* (greater than/less than), *Change detection* (increases/decreases), *Range monitoring*, and *Missing data* (to catch sensor failures).

- **Occurrence:** You can trigger the rule *Every time* the condition is met or only when it has been true for a specific duration (to avoid reacting to transient issues).

3. Focus Your Scope

The **Property Filter** section allows you to narrow the rule's application so it doesn't evaluate every event in the stream.

- You can apply up to **three filters** to target specific data, such as only monitoring "high-priority" shipments or specific cities.

Unit 4 Configure actions in Activator

Unit 4 focuses on [Configure actions in Activator](#), which are the final steps that turn data detection into meaningful responses.

Activator provides four primary types of actions:

- **Email Actions:** Best for sending detailed context that doesn't require an instant response (e.g., daily status reports or non-critical threshold alerts).
- **Teams Actions:** Used for high-priority alerts that require immediate team coordination and discussion.
- **Power Automate Actions:** Triggers multi-step workflows across external apps and services, allowing for complex business process automation.
- **Fabric Item Actions:** Executes internal Fabric components like **Notebooks** or **Data Pipelines** for advanced data processing or system adjustments.

Implement a Data Warehouse

Introduction to end-to-end analytic

Get started with data warehouses

Unit 1 Introduction

Get started with data warehouses in Microsoft Fabric:

- **Core Role:** Relational data warehouses serve as the foundation for enterprise business intelligence (BI), providing a structured, SQL-based environment to store and analyze large-scale business data.
- **Microsoft Fabric Capability:** Fabric offers a fully managed data warehouse with full transactional T-SQL support (including **INSERT**, **UPDATE**, **DELETE**, and **MERGE** operations).
- **OneLake Integration:** Data is stored in the open **Delta format** on OneLake, allowing seamless integration across other Fabric workloads and providing an intelligent data foundation for Copilot and AI-driven insights.
- **What the Module Covers:** * Fundamentals of dimensional modeling using **fact and dimension tables**.
 - Fabric-specific features like Copilot-assisted query authoring.
 - Data querying, table structuring, security, and monitoring for governance and performance.

Unit 2 Understand data warehouses

Core Definition & Process

A data warehouse is a centralized, structured store optimized for analytical queries and reporting rather than day-to-day transactions. Building one typically involves:

- **Data Ingestion:** Moving data from source systems.
 - **Data Storage:** Storing data in an analytics-optimized format.
 - **Data Processing:** Transforming data for analytical tools.
 - **Data Analysis & Delivery:** Gaining and delivering business insights.
-

Dimensional Modeling (Tables)

Data is organized using **dimensional modeling** to support efficient analysis of large data volumes through two main types of tables:

- **Fact Tables:** Contain the primary numerical data and metrics you want to analyze (e.g., total sales revenue). They usually have a large number of rows.
- **Dimension Tables:** Provide descriptive context for the data in fact tables (e.g., customer details, store locations). They use two types of keys:

- **Surrogate Key:** A unique identifier internal to the data warehouse for consistency.
- **Alternate Key:** A natural or business key (like a product code) to maintain traceability back to the source system.

Specialized Dimensions

- **Time Dimensions:** Enable analysts to aggregate data over specific intervals (years, quarters, months, days).
 - **Slowly Changing Dimensions (SCD):** Track changes to attributes over time (e.g., a customer changing their address) to ensure accurate historical reporting.
-

Schema Designs

Unlike transactional databases that normalize data to reduce duplication, data warehouses use **denormalized** data to minimize complex query joins.

- **Star Schema:** The most common design where a central fact table connects directly to surrounding dimension tables. Microsoft provides specific design guidance in [What is a star schema?](#) for Fabric environments.
- **Snowflake Schema:** An extension of the star schema where dimension tables are further split (normalized) into secondary dimension tables to handle complex hierarchies (e.g., breaking a product table down into categories and suppliers).

Unit 3 Understand data warehouses in Fabric

What is a Fabric Data Warehouse?

A Fabric data warehouse is a fully managed, enterprise-scale relational database built on **OneLake**. It features full transactional T-SQL support (DDL and DML) with complete ACID compliance, storing its data in the open **Delta format** so other Fabric workloads can access it without duplication.

Warehouse vs. SQL Analytics Endpoint

Fabric workspaces include two different SQL-based items depending on your needs:

- **Warehouse:** Supports full **read and write** T-SQL capabilities (**INSERT**, **UPDATE**, **DELETE**, **MERGE**) and handles native warehouse tables.

- **SQL Analytics Endpoint:** A **read-only** SQL experience used to query Lakehouse Delta tables.
-

Key Data Warehouse Operations

1. Ingestion Methods

Data can be brought into the warehouse using several approaches:

- **COPY INTO:** Bulk loads data directly into tables from external files (like CSV or Parquet) in Azure storage.
- **OPENROWSET:** Queries files directly from external storage or OneLake for ad hoc analysis without needing to create a table first.
- **Pipelines and Dataflows:** Uses Data Factory tools for automated, orchestrated data movement.
- **Cross-Database Queries:** Allows you to query tables in a lakehouse directly from your warehouse using a three-part naming convention (`database.schema.table`), completely avoiding data duplication.

2. Using Staging Tables

A standard data warehousing pattern involves landing raw data into **staging tables** (which mirror the source data structure) before transforming and loading it into the final dimension and fact tables. This keeps the source data intact while you apply business logic and lookups.

3. Zero-Copy Table Clones

Fabric allows you to create **table clones** using T-SQL. Clones copy the table's metadata but reference the exact same underlying data files in OneLake. Because the data isn't duplicated, this keeps storage costs low and is highly useful for testing, development, and historical reporting.

Unit 4 Query and transform data

Querying Tools in Fabric

Fabric provides two distinct query environments to accommodate different skill sets:

- **SQL Query Editor:** A code-first experience for writing T-SQL, featuring IntelliSense, code completion, and syntax highlighting (similar to SSMS or Azure Data Studio). It also features **Copilot for Data Warehouse** to generate, complete, explain, or fix queries using natural language.

- **Visual Query Editor:** A no-code, drag-and-drop interface similar to the [Power Query online diagram view](#). Users can drag tables onto a canvas and use a transformation menu to filter, aggregate, and add columns.
-

Reusable Transformation Objects

To standardize data access and handle repeatable data shaping tasks, you can save your logic directly in the warehouse:

- **Views:** Reusable, saved queries referenced like standard tables. They are used to simplify data for analysts (e.g., pre-joining fact and dimension tables). Well-named views also help AI tools, like Copilot and Fabric IQ data agents, query the warehouse more accurately using natural language.
- **Stored Procedures:** Executable T-SQL scripts used to automate operational data tasks, such as applying complex business rules or moving data from staging areas into final reporting tables.

Unit 5 Model data in a warehouse

1. Prepare Data for Consumption

Raw database tables often contain technical clutter (like staging tables, surrogate keys, and internal ETL flags) that creates noise for business users. To optimize the consumer experience in the model view:

- **Hide internal objects** that aren't necessary for analysis.
- **Rename columns** to use clear, business-friendly terms (e.g., changing `CustRgn` to `Customer Region`).
- **Add clear descriptions** to tables and columns.

 **AI Optimization:** Keeping your data model clean and well-documented directly improves the accuracy of Copilot in Power BI and Fabric IQ data agents when they translate natural language questions into SQL or DAX.

2. Define Table Relationships

Relationships encode the logical connections between tables so users don't have to write explicit `JOIN` logic every time they query. In a standard star schema, these connections map fact tables to dimension tables using shared key columns.

- **Cardinality:** Fact-to-dimension relationships are typically **many-to-one** (multiple fact rows mapping to a single dimension row).
- **Cross-filter direction:** Usually set to a **single direction** (where the dimension filters the fact table) to ensure predictable and performant behavior.

For deeper guidance on structuring these connections, you can refer to the module on how to [Design dimensional models in Microsoft Fabric](#).

3. Standardize Access with Views and Measures

To prevent different teams from writing conflicting formulas or join logic, you can embed a single source of truth directly into the warehouse:

- **Views:** Encapsulate T-SQL join logic, filters, and column selections into reusable queries that behave like tables. They provide a stable data source for reports even if the underlying base tables change.
 - **Measures:** Reusable DAX expressions that standardize business calculations (like total sales, averages, or counts) for reporting. Updating a measure definition in the model view instantly applies the change across all downstream reports.
-

4. Create a Semantic Model for Power BI

Once the data is cleaned, connected, and standardized, it is ready for reporting. For interactive Power BI dashboards, you can create a semantic model that utilizes **Direct Lake mode**.

Unlike traditional import modes that duplicate data into Power BI memory, Direct Lake reads data directly from the native Parquet files in OneLake. This ensures your reports always reflect the latest warehouse data without the overhead of scheduled data refreshes or separate storage maintenance. To explore scalability options for these models, see the guide on how to [Design scalable semantic models](#).

Unit 5 Secure and monitor a warehouse

Security

Fabric uses a multi-layered security model to balance data democratization with strict governance. Security policies defined in T-SQL are automatically enforced regardless of how the data is accessed (including by AI tools like Copilot).

Access Control Layers

- **Workspace Roles:** The primary layer of access control (e.g., *Admins* have full control, while *Viewers* can only see items without making changes). Learn more about configuring these in the [Workspaces in Power BI](#) documentation.
- **Item Permissions:** Allows sharing of individual warehouses without granting access to the entire workspace. Key granular permissions include:
 - **Read:** Allows connecting via the SQL analytics endpoint.
 - **ReadData:** Allows reading data from any table or view.
 - **ReadAll:** Allows reading raw Parquet files directly in OneLake.

Granular SQL Security

For precise, internal table security without altering underlying data, Fabric supports standard T-SQL features:

- **Object-level security:** Controls access to specific tables, views, or stored procedures.
- **Row-level security (RLS):** Restricts row visibility using **WHERE** clause predicates.
- **Column-level security (CLS):** Restricts visibility on specific columns.
- **Dynamic data masking:** Obscures sensitive data (like emails or account numbers) from non-privileged users.

For a deeper dive into these T-SQL features, refer to the guide on how to [Secure a Microsoft Fabric data warehouse](#).

Monitoring

Monitoring tracks query performance, identifies bottlenecks, and analyzes resource usage through two primary methods:

1. Query Insights

A central historical repository that retains data for 30 days. It uses system views that you can query directly to find long-running or resource-intensive queries:

- **queryinsights.exec_requests_history** (completed SQL requests)
- **queryinsights.long_running_queries** (queries ranked by execution time)
- **queryinsights.exec_sessions_history** (completed sessions)

2. Dynamic Management Views (DMVs)

Used for real-time monitoring of active connections and running sessions. For example, you can query **sys.dm_exec_requests** to see live, active queries.

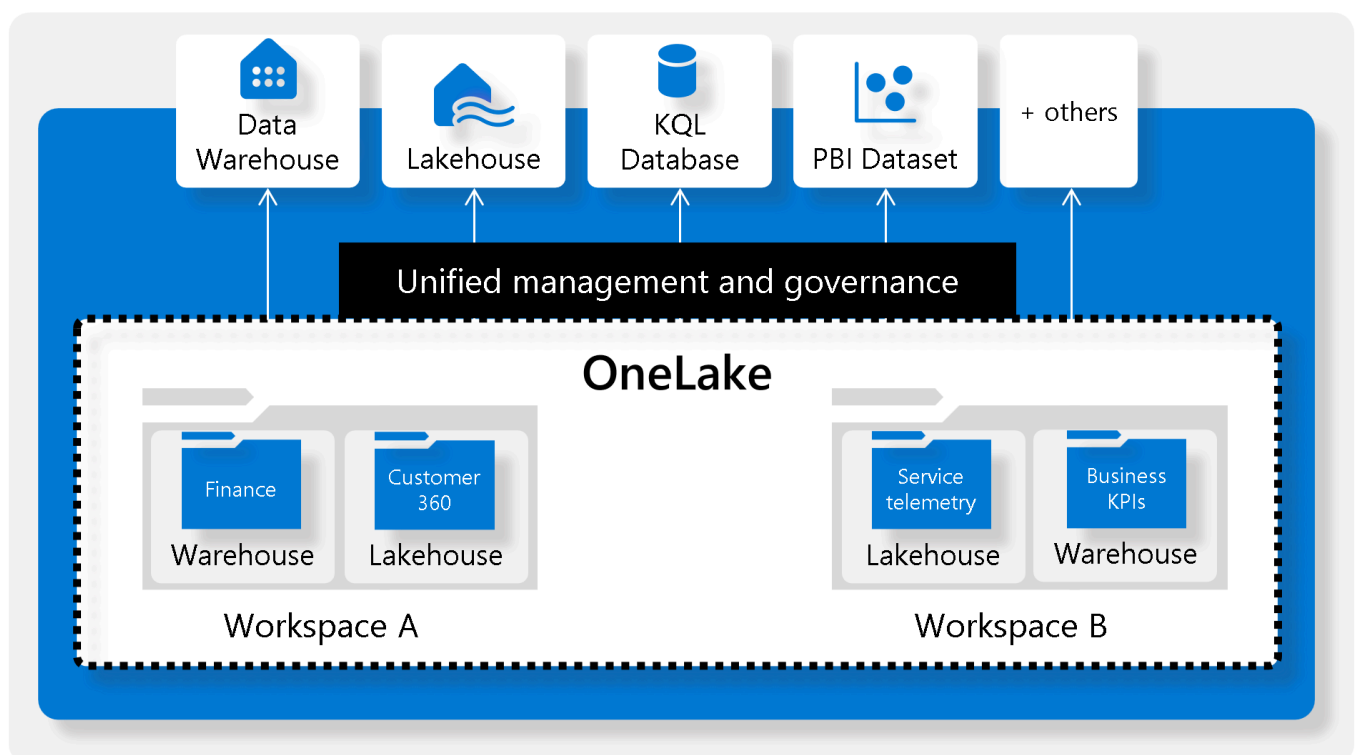
 **Note:** While Members, Contributors, and Viewers can monitor their own queries, you must be a workspace **Admin** to see other users' queries or run the **KILL** command to terminate long-running sessions.

Load data into warehouse

Unit 1 Introduction

Microsoft Fabric Data Warehouse Overview

- **Unified Platform:** [Microsoft Fabric Data Warehouse](#) is a complete platform for data, analytics, and AI designed to store, organize, and manage large volumes of structured and semi-structured data.
- **T-SQL Support:** It provides advanced query processing and fully supports transactional T-SQL capabilities, similar to an enterprise data warehouse.
- **OneLake Integration:** Unlike Synapse Analytics' dedicated SQL pools, a Fabric warehouse centers around a single data lake. Data is stored in the **Parquet file format** within [Microsoft OneLake](#), allowing users to leverage a single copy of their data across data preparation, analysis, and reporting tasks.



The ETL (Extract, Transform, and Load) Process

The core foundation for data analytics and warehousing in Fabric relies on the ETL framework:

- **Data Extraction:** Connecting to source systems and gathering the necessary data.
- **Data Transformation:** Cleaning, deduplicating, validating, and combining data from different tables to convert it into a standardized format.

- **Data Loading:** Feeding the transformed data into fact and dimension tables (including periodic incremental loads that reformat data for schema compatibility).
- **Post-load Optimizations:** Enhancing the performance of the data warehouse once the data is safely stored.

Note: These ETL steps can run in parallel. As soon as a portion of data is processed and ready, it can be loaded immediately without waiting for previous steps to fully complete.

Unit 2 Explore data load strategies

Data Ingestion vs. Data Loading

While both are key parts of the ETL (Extract, Transform, Load) pipeline, they serve distinct purposes:

- **Data Ingestion (Extract):** The process of moving raw data from various source systems into a central repository.
- **Data Loading:** Taking that transformed, clean data and loading it into the final storage destination for analysis.
- **Storage Format:** Fabric data warehouses and lakehouses automatically store this data in [Microsoft OneLake](#) using the **Delta Parquet** format.

Data Staging

- **Purpose:** Staging involves creating auxiliary objects (like temporary tables, stored procedures, and functions) to act as a buffer.
- **Benefit:** It abstracts and simplifies the final load operation while minimizing the performance impact on the active data warehouse.

Types of Data Loads

The choice between load types depends on data volume, characteristics, and business requirements:

Load Type	Description	Operation	Complexity & Duration
Full (Initial) Load	Populates the data warehouse for the	Tables are truncated and completely	Simpler to implement but

	first time or performs a complete refresh.	reloaded; old data is lost.	takes longer due to data volume.
Incremental Load	Regularly updates the warehouse with changes since the last update.	Tables are updated with new info; historical data is preserved.	More complex to implement but completes much faster.

Keys and Table Loading Strategies

Keys

- **Business (Natural) Key:** An identifier from the source system that holds business meaning (e.g., a product code or customer ID). It ensures traceability back to the source.
- **Surrogate Key:** A system-generated, unique identifier (usually an integer) with no business meaning. It maintains data consistency even if source system business keys change or get reused.

Loading Dimension Tables (The Context)

Dimension tables provide the descriptive attributes (the "who, what, where, why") for your data. They often handle **Slowly Changing Dimensions (SCD)** to manage data that changes unpredictably over time:

- **Type 0 SCD:** The dimension attributes never change.
- **Type 1 SCD:** Overwrites existing data, doesn't keep history.
- **Type 2 SCD:** Adds new records for changes, keeps full history for a given natural key.
- **Type 3 SCD:** History is added as a new column.
- **Type 4 SCD:** A new dimension is added.
- **Type 5 SCD:** When certain attributes of a large dimension change over time, but using type 2 isn't feasible due to the dimension's large size.
- **Type 6 SCD:** Combination of type 2 and type 3.

Loading Fact Tables (The Measurements)

Fact tables store numerical metrics and are typically loaded *after* dimension tables. The loading process must look up the business keys from incoming data and map them to the correct surrogate keys in the dimension tables, ensuring historical alignment at the exact time the event occurred.

Unit 3 Use data pipelines to load a warehouse

What is a Fabric Data Pipeline?

- **Definition:** A cloud-based data integration service used to build scalable workflows for moving and transforming data.
 - **Foundation:** Built primarily on **Azure Data Factory** functionality, it allows users to orchestrate complex ETL/ELT processes visually.
 - **Best For:** Code-free or low-code experiences, scheduled workflows, and connecting to disparate data sources.
 - **Storage Note:** All ingested data is automatically stored in the **Delta Parquet** format within OneLake.
-

Creating a Pipeline

You can open the data pipeline editor in two ways:

1. **From the Workspace:** Select **+ New > Data pipeline**.
2. **From the Warehouse Asset:** Select **Get Data > New data pipeline**.

When you start a new pipeline, you are presented with three creation paths:

- **Add pipeline activity:** Opens a blank canvas to build a custom pipeline from scratch.
 - **Copy data:** Opens a step-by-step assistant to quickly move data from a source to a destination.
 - **Choose a task to start:** Provides predefined templates for common data movement scenarios.
-

The Copy Data Assistant

The assistant guides you through a wizard to set up a data movement task without writing code:

- **Choose & Connect to Source:** Select a data connector, enter credentials, and select tables/views (or write a custom query) to preview the data.
- **Choose & Connect to Destination:** Select the target data store and map source columns directly to new or existing destination tables.

- **Settings:** Configure advanced details like staging environments and default data values.
-

Scheduling and Automation

Pipelines can be fully automated to run on a set schedule. This can be configured directly within the pipeline editor by selecting **Schedule** or by navigating to **Settings** in the Home menu.

Unit 4 Load data using T-SQL

Overview

Microsoft Fabric's Warehouse is powered by the same enterprise-grade SQL engine that database developers are already familiar with. This allows users to use standard T-SQL for complex data manipulations, including filtering, sorting, aggregating, and joining data across different tables directly at the database level.

The **COPY** Statement

The [COPY statement](#) is the primary method for efficiently importing bulk data into the warehouse from an external Azure storage account (using Shared Access Signature or Storage Account Key authentication).

- **Flexibility:** It supports multiple file types (such as CSV and Parquet) and configuration options like skipping header rows or using wildcards to ingest multiple files at once.
 - **Error Handling:** For CSV files, you can isolate bad data by defining a separate **ERRORFILE** location (**REJECTED_ROW_LOCATION**) to catch and inspect rows that fail to import.
 - **Structural Requirement:** All source files must share the exact same column structure and order as the target warehouse table.
-

Loading Data Across Warehouses and Lakehouses

Because all data assets within the same Fabric workspace share a single logical SQL server, you can easily perform cross-database queries.

- **Three-Part Naming:** You can reference disparate data assets using the standard `[Asset_Name].[Schema_Name].[Table_Name]` convention.
- **Cross-Asset Joins:** Analysts can combine and consolidate data from multiple warehouses and lakehouses into a single target table.
- **T-SQL Methods:** Data can be manipulated and loaded using standard commands:
 - **CREATE TABLE AS SELECT (CTAS):** Creates a brand new table based on the results of a complex `SELECT` query.
 - **INSERT...SELECT:** Appends data from a source query into an already existing target table.
- **Tooling:** These cross-database operations can be run directly inside the Fabric workspace visual query editor or using familiar external SQL clients like SQL Server Management Studio (SSMS).

Unit 5 Load and transform data with Dataflow Gen2

Overview of Dataflow Gen2

- **Definition:** [Dataflow Gen2](#) is the next generation of dataflows in Microsoft Fabric, offering a streamlined Power Query experience that reduces the number of steps required to import, clean, and reshape data.
 - **Use Cases:** It can be used within data pipelines to ingest data into a lakehouse or warehouse, or to prepare a dataset for Power BI reports.
-

Data Ingestion & Transformation

- **Importing Data:** Users can quickly load various file types (such as text or CSV files from a local computer) through the interface.
 - **Copilot Integration:** AI-powered Copilot can automate transformations. Users can provide natural language instructions (e.g., *"Transform the Gender column. If Male 0, if Female 1."*) and Copilot will automatically apply the steps, which can be reviewed or reverted if necessary.
-

Data Destinations & Update Methods

By separating ETL logic from destination storage, Dataflow Gen2 allows for cleaner, more maintainable data pipelines. Supported destinations include:

- Azure SQL Database
- Lakehouse
- Azure Data Explorer (Kusto)
- Azure Synapse Analytics (SQL DW)
- Warehouse

Warehouse Update Options

When loading data specifically into a warehouse target, you can choose between two update methods:

- **Append:** Adds new rows to an existing destination table.
 - **Replace:** Completely wipes out the existing content of the table and replaces it with the newly transformed dataset.
-

Publishing

The final step in the process is publishing the dataflow. Publishing makes the transformations live, allowing the dataflow to be executed manually or automated on a recurring schedule. Any subsequent modifications to the dataflow will only take effect once it is published again.

Query a data warehouse

Unit 1 Introduction

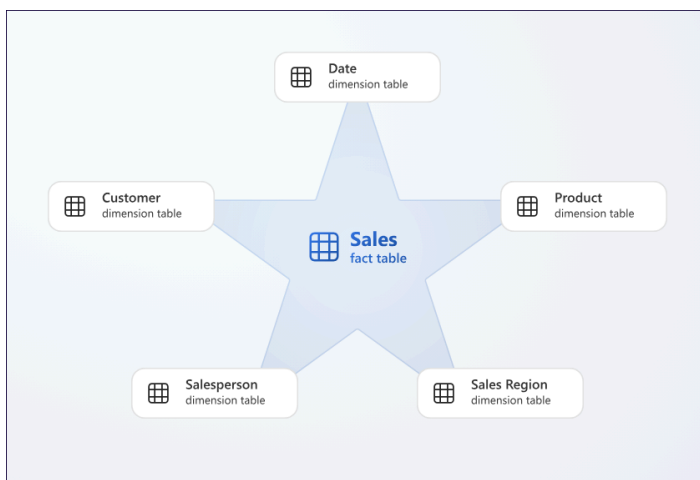
Microsoft Fabric Data Warehouse Overview

- **All-in-One Platform:** A complete environment for data, analytics, and AI designed to manage large volumes of both structured and semi-structured data.
 - **T-SQL Capabilities:** Offers robust query processing and supports full, transactional T-SQL capabilities identical to an enterprise relational data warehouse.
 - **Business Intelligence Role:** Querying the warehouse allows organizations to extract and analyze data, turning massive amounts of raw storage into actionable business insights.
-

Star Schema Design

Data warehouses typically organize data using relational modeling approaches like a **star schema** or a **snowflake schema**. This involves classifying data into two distinct types of tables:

- **Fact Tables:** Store the core quantitative, measurable data about a business (e.g., quantities sold, revenue, or transaction metrics).
- **Dimension Tables:** Contain descriptive attributes that provide context to those facts. They answer the operational questions surrounding an event, such as:
 - **Customer:** *Who* bought it?
 - **Product:** *What* was bought?
 - **Store / Sales Region:** *Where* was it bought?
 - **Date:** *When* was it bought?
 - **Currency:** *How* was it valued?



Key Takeaway: Without dimension tables to provide context, the raw numbers inside a fact table have no business meaning.

Unit 2 Query data

Aggregating Measures by Dimension Attributes

A standard querying pattern in data warehousing involves joining a fact table to one or more dimension tables to aggregate numeric data (like sales amounts) grouped by descriptive attributes (like year, quarter, or city).

- **Star Schema Joins:** Easily group data by joining primary keys to foreign keys (e.g., joining `FactSales` to `DimDate` and `DimCustomer`).
 - **Snowflake Schema Joins:** If dimensions are normalized (e.g., categories are split out from products into a separate `DimCategory` table), you must traverse the entire relationship chain using multiple `JOIN` statements, even if intermediate tables aren't displayed in the final results.
-

Using Ranking Functions

T-SQL ranking functions allow you to partition data across categories and return a value indicating each row's relative position within that partition:

- **`ROW_NUMBER()`:** Returns the sequential, ordinal position of a row within the partition (1, 2, 3, etc.).
 - **`RANK()`:** Returns the ranked position. If there is a tie, rows receive the same rank, but subsequent ranks are skipped (e.g., 1, 2, 3, 3, 5).
 - **`DENSE_RANK()`:** Functions like `RANK()`, but ties do not create gaps in the subsequent numbering (e.g., 1, 2, 3, 3, 4).
 - **`NTILE(x)`:** Divides the partitioned rows into a specified number of ranked groups or percentiles (e.g., `NTILE(4)` groups data into quartiles).
-

Retrieving an Approximate Count

When performing initial data exploration on massive datasets, calculating precise distinct counts using standard `COUNT(DISTINCT ...)` functions can take a significant amount of time.

- **`APPROX_COUNT_DISTINCT`:** Uses the *HyperLogLog* algorithm to quickly estimate the number of unique values.
- **Accuracy Trade-off:** It guarantees a maximum error rate of 2% with 97% probability. While less accurate, it runs significantly faster and is ideal for broad high-level data comparisons.

Unit 3 Use the SQL query editor

Overview

The SQL query editor in Microsoft Fabric allows you to write and run T-SQL scripts directly against your data warehouse. It includes built-in IntelliSense, syntax highlighting, and debugging support, eliminating the need for external connection configuration.

Launching and Running Queries

- **Creation Paths:** You can start a new query from the warehouse *Explorer* via the **Home menu** (selecting *New SQL query* or a template) or by right-clicking the **Queries node** under *My queries*.
 - **Auto-Save:** Queries are automatically saved in your workspace as you work.
 - **Execution:** Typing a script and clicking **Run** executes the query. The *Results* pane displays a preview of the data, capped at a maximum of 10,000 rows.
-

Working with Query Results

The results toolbar provides several built-in features to manage and analyze your output:

- **Open in Excel:** Establishes a live, refreshable connection to Microsoft Excel rather than downloading a static file.
 - **Explore this data (preview):** Opens a side-by-side visual and matrix view to spot data trends ad hoc.
 - **Visualize results:** Directly generates a Power BI visual from the query results (note: this feature does not support queries using an **ORDER BY** clause).
 - **Data Extraction:** Results can be copied (with or without column names), and a dropdown menu lets you navigate between multiple result sets.
 - **Schema Creation:** You can directly save your highlighted **SELECT** statement as a permanent **view** or a new **table** within the warehouse.
-

Copilot Integration

If enabled in your tenant, Copilot offers quick-action buttons directly in the query ribbon:

- **Explain query:** Automatically adds natural language comments explaining what each part of your T-SQL script does.

- **Fix query errors:** Activates when an error is detected to automatically repair syntax or logic issues.
- **Chat Pane:** Allows you to open a dedicated chat panel to generate T-SQL scripts using natural language prompts.

⚠ **Note:** Copilot capabilities require an F2 or P1 capacity or higher and are not available on trial SKUs. For more details, you can review the [documentation on Copilot for Data Warehouse](#).

Unit 4 Explore the visual query editor

Overview of the Visual Query Editor

The [visual query editor](#) in Microsoft Fabric provides an intuitive, graphical, no-code interface for building data warehouse queries.

- **Graphical Interface:** Users can drag and drop tables from the *Explorer* pane directly onto a visual design canvas. The editor visually maps out relationships between tables, making complex structures easy to understand at a glance.
 - **Automatic Query Generation:** As you manipulate elements on the canvas, the tool automatically writes the underlying T-SQL script in the background.
-

Building Queries Visually

- **Merging/Joining Data:** Once tables are placed on the canvas, they can be joined by right-clicking a table and selecting **Merge queries**. A configuration dialog allows you to choose matching key columns and define the specific join type.
 - **Executing:** Clicking **Run** executes the visual logic and displays the results in a preview pane below the canvas.
-

Results and Downstream Options

The visual query editor shares the same action toolbar found in the standard SQL query editor. From the results pane, users can:

- Save the output as a permanent **table** or **view**.
 - Export the data by downloading a live **Excel file**.
 - Instantly **visualize results** as a chart.
-

Unit 5 Use client tools to query a warehouse

Connecting with SQL Server Management Studio (SSMS)

You can connect external tools like [SSMS](#) directly to your Microsoft Fabric data warehouse to manage and query data.

- **Step 1: Get the Connection String:** In your Fabric workspace, click the ellipsis (. . .) next to your warehouse name and select **Copy SQL connection string**.
 - **Step 2: Configure SSMS:** * Paste the string into the **Server name** field.
 - Explicitly enter your warehouse name in the **Database Name** field (leaving this blank may cause the connection to fail).
 - **Step 3: Authenticate:** Select your organization's Microsoft Entra method and connect.
-

Authentication Requirements

- **Microsoft Entra ID Only:** Microsoft Fabric exclusively supports Microsoft Entra ID authentication methods.
 - **No SQL Authentication:** Traditional SQL Server username/password authentication is **not** supported.
-

Using Other Third-Party Tools

- **Universal Compatibility:** Any third-party data tool can connect to a Fabric Warehouse or SQL analytics endpoint.
- **Drivers:** This is achieved using standard ODBC or OLE DB drivers combined with Microsoft Entra ID authentication.
- **Network Requirement:** To allow external traffic, **TCP port 1433** must be left open in your network firewall.

Accessing the Code

If you want to transition from a no-code experience to a pro-code experience, you can select **View SQL** in the ribbon to inspect the automatically generated T-SQL, or select **Edit SQL script** to open and modify the code directly within the traditional SQL query editor

Monito Data Warehouse

Monitor a Microsoft Fabric data warehouse

Unit 1 Introduction

Critical Asset: A data warehouse serves as the central hub for enterprise analysis and reporting, making its reliability vital to the business.

Key Monitoring Goals: * Track and manage operational **costs**.

- Identify and resolve **query performance** bottlenecks.
- Gain deep insights into **data usage** patterns.

Unit 2: Monitor capacity metric

Core Concepts

- **Capacity and Cost:** Your Microsoft Fabric license determines your available **capacity** (a pool of computing resources). Operations are billed based on **Capacity Units (CUs)**.
- **Warehouse Consumption:** In data warehousing, CUs are primarily consumed by data read/write activities, queries, and file operations within OneLake storage. Monitoring this usage is essential for cost management and planning.

The Microsoft Fabric Capacity Metrics App

Administrators can install this specialized app to monitor resource utilization trends. Key capabilities include:

- **Targeted Filtering:** The interface can be filtered to display only *warehouse* activity, isolating data warehouse workloads from other Fabric tasks.
- **Identifying Bottlenecks:** The app helps detect **throttling**, which occurs when your processes demand more computing power than your purchased capacity license allows.
- **Optimization:** By tracking these utilization trends, organizations can properly right-size and optimize their capacity licensing needs.

Unit 3: Monitor current activity

Core Concepts

- **Dynamic Management Views (DMVs):** Microsoft Fabric data warehouses include built-in system views that you can query using T-SQL to retrieve real-time data about the current state and live activities of the warehouse.

- **The Key DMVs:**

- `sys.dm_exec_connections`: Provides details about active data warehouse connections.
- `sys.dm_exec_sessions`: Contains information regarding authenticated user sessions.
- `sys.dm_exec_requests`: Details currently active, executing requests.

Practical Application

- **Identifying Bottlenecks:** By joining these three DMVs together in a SQL query, database administrators can isolate exactly who is running a query, what command they are executing, and how long it has been running.
- **Optimization:** Sorting this data by elapsed time allows teams to quickly spotlight and troubleshoot long-running queries that are hogging resources and could benefit from performance optimization.

Querying DMVs

You can retrieve detailed information about current activities in the data warehouse by querying the `dm_exec-*` DMVs. For example, consider the following query:

SQL

Copy

```
SELECT sessions.session_id, sessions.login_name,
       connections.client_net_address,
       requests.command, requests.start_time, requests.total_elapsed_time
FROM sys.dm_exec_connections AS connections
INNER JOIN sys.dm_exec_sessions AS sessions
  ON connections.session_id=sessions.session_id
INNER JOIN sys.dm_exec_requests AS requests
  ON requests.session_id = sessions.session_id
WHERE requests.status = 'running'
  AND requests.database_id = DB_ID()
ORDER BY requests.total_elapsed_time DESC
```

This query returns details about the active requests in the current database, ordered by the duration for which they have been executing; which may be useful to identify long-running queries that could benefit from optimization. An example result set from the query is shown here:

Expand table

session_id	login_name	client_net_address	command	start_time
60	fred@contoso.com	10.23.139.162	SELECT	2023-12-07T14:56:41.3530000
126	nandita@contoso.com	10.23.137.98	SELECT	2023-12-07T14:57:22.7800000
137	zoe@contoso.com	10.23.119.171	SELECT	2023-12-07T14:57:38.6070000

Unit 4: Monitor queries

Core Concepts

- **Query Insights Feature:** Microsoft Fabric data warehouses provide historical, aggregated information about executed queries. This feature helps database administrators tune query performance by identifying heavily used or slow-running commands.
- **Key System Views:** Fabric tracks this data using three specialized views under the `queryinsights` schema:
 - `queryinsights.exec_requests_history`: Contains details for every completed SQL query. *(Note: It can take up to 15 minutes for new queries to appear here).*
 - `queryinsights.long_running_queries`: Tracks execution time metrics to spotlight performance bottlenecks.
 - `queryinsights.frequently_run_queries`: Tracks execution counts, showcasing how often specific queries are utilized.

Practical Application

- **Analyzing Resource Trends:** You can run standard T-SQL queries against these views to extract actionable performance data.
- **Query Parameterization:** To ensure aggregated statistics remain accurate, Fabric automatically parameterizes queries with predicates. This means queries looking for different specific dates (e.g., `WHERE orderdate > '01/01/2023'` vs. `WHERE orderdate > '12/31/2021'`) are recognized and grouped as the exact same command.

Secure a Microsoft Fabric data warehouse

Secure a Microsoft Fabric data warehouse

Unit 1 Introduction

Overview of Fabric Warehouse Security

Microsoft Fabric Data Warehouse provides built-in security features powered by its underlying SQL engine. These measures safeguard sensitive information across both **Warehouses** and **SQL analytics endpoints** automatically, meaning no changes are required for existing downstream applications.

Core Security Layers

- **Workspace Roles:** Manages high-level access and control within the workspace environment by assigning users specific roles: *Admin*, *Member*, *Contributor*, or *Viewer*.
- **Item Permissions:** Permissions applied directly to individual warehouses, primarily used to facilitate secure sharing for downstream utilization.
- **Data Protection Security (Granular Control):** Administrators and data engineers can use T-SQL to implement advanced, fine-grained control over sensitive data, including:
 - **Object-level security:** Securing database objects.
 - **Column-level security:** Restricting access to specific table columns.
 - **Row-level security:** Filtering table rows using **WHERE** clauses based on user context.
 - **Dynamic data masking:** Obscuring sensitive data fields (such as email addresses) automatically.

Unit 2 Explore dynamic data masking

What is Dynamic Data Masking (DDM)?

[Dynamic Data Masking \(DDM\)](#) is a security feature that protects sensitive data by obscuring it in query results for non-privileged users.

Key Benefits

- **Real-Time Masking:** Data is masked on the fly when a query is executed.
- **Preserves Underlying Data:** The actual data in the database remains completely unchanged and intact.

- **Easy Implementation:** It is configured at the column level via T-SQL and requires no complex coding or modifications to application queries.

Masking Functions & Rules

Administrators can apply different masking types depending on the data type and use case:

Masking Type	Rule Syntax	Description / Use Case
Default	<code>default()</code>	Completely masks the data based on the column's data type. No information is visible.
Email	<code>email()</code>	Exposes only the first letter of the email address and the constant suffix <code>.com</code> (e.g., <code>j*****@contoso.com</code>).
Custom Text	<code>partial(prefix, padding, suffix)</code>	Pads the middle of a string while leaving a specified number of starting and ending characters visible.
Random	<code>random(low, high)</code>	Replaces numeric or binary values with a random number within a designated range.

Configuration Example

To apply masking to customer personal information, you use the `ALTER TABLE` statement in Transact-SQL:

SQL

-- Masking an Email column

ALTER TABLE Customers

ALTER COLUMN Email ADD MASKED WITH (FUNCTION = 'email()');

-- Masking a Phone Number to expose only the last 4 digits

```
ALTER TABLE Customers
```

```
ALTER COLUMN PhoneNumber ADD MASKED WITH (FUNCTION =  
'partial(0,"XXX-XXX-",4)');
```

 **Security Note:** DDM is not a physical encryption method, and unprivileged users with heavy query permissions could potentially infer the underlying data. It should always be paired with **Object-Level Security (OLS)** and the principle of least privilege as part of a broader security strategy.

Unit 3 Implement row-level security

What is Row-Level Security (RLS)?

[Row-Level Security \(RLS\)](#) provides fine-grained control over access to specific rows in a database table. Access is determined dynamically based on the executing user's context, group membership, or identity (such as their username).

Key Characteristics

- **Transparent to Users:** SQL Server enforces the security rules automatically on the backend, meaning rows a user isn't permitted to see simply do not appear in their query results.
- **Minimal Application Changes:** Because security policies are managed directly at the warehouse level, downstream applications do not need complex recoding to filter data.
- **Granular Restraints:** Filters restrict **SELECT**, **UPDATE**, and **DELETE** operations (though **INSERT** is not affected by default filter predicates).

How RLS is Configured

Implementing RLS requires a two-step process using T-SQL:

1. **Filter Predicate:** An inline table-valued function that evaluates data rows. It returns **true** (allowing access) or **false** (blocking access) based on a conditional statement (e.g., matching the user's login with a column value).
2. **Security Policy:** A database policy that binds the filter predicate function to the target table and turns the enforcement state **ON** or **OFF**.

T-SQL Implementation Example

SQL

-- 1. Create a dedicated security schema

```
CREATE SCHEMA [Sec];
```

```
GO
```

```
-- 2. Create the filter predicate function
CREATE FUNCTION sec.tvf_SecurityPredicatebyTenant(@TenantName AS
NVARCHAR(10))
RETURNS TABLE WITH SCHEMABINDING
AS
RETURN SELECT 1 AS result
WHERE @TenantName = USER_NAME() OR USER_NAME() =
'tenantAdmin@contoso.com';
GO

-- 3. Create and enable the security policy on a table
CREATE SECURITY POLICY sec.SalesPolicy
ADD FILTER PREDICATE sec.tvf_SecurityPredicatebyTenant(TenantName)
ON [dbo].[Sales] WITH (STATE = ON);
GO
```

Use Cases & Best Practices

Common Use Cases

- **Multi-tenant Isolation:** Restricting corporate customers so they can only see data belonging to their own company.
- **Departmental Segregation:** Limiting internal staff access to rows relevant only to their specific division or region.
- **Compliance:** Enforcing legal or regulatory constraints on data access.

Best Practices

- **Schema Isolation:** Always house security predicate functions and security policies in a separate, dedicated database schema.
- **Performance Optimization:** Avoid data type conversions, excessive table joins, and recursive logic inside your predicate functions to prevent query slowdowns.
- **Side-Channel Risk Awareness:** Be mindful of "side-channel attacks," where malicious users craft specific query errors (like a divide-by-zero) to infer whether hidden row conditions evaluate to true or false.

Unit 3 Implement column-level security

What is Column-Level Security (CLS)?

[Column-Level Security \(CLS\)](#) allows organizations to restrict access to specific, sensitive columns within a database table based on a user's role or execution context. This provides granular protection for sensitive data (such as medical records or financial information) without requiring changes to table structures.

How CLS is Configured

CLS is managed directly within the database using standard T-SQL commands (**GRANT** and **DENY**).

T-SQL Implementation Example

To protect a sensitive column (like **MedicalHistory**) in a patient database, you define database roles, grant broad table access, and explicitly deny access to specific columns for restricted roles:

```
SQL
-- 1. Create database roles
CREATE ROLE Doctor AUTHORIZATION dbo;
CREATE ROLE Receptionist AUTHORIZATION dbo;
GO

-- 2. Grant general SELECT access on the table to all roles
GRANT SELECT ON dbo.Patients TO Doctor;
GRANT SELECT ON dbo.Patients TO Receptionist;
GO

-- 3. Explicitly DENY access to the sensitive column for the Receptionist role
DENY SELECT ON dbo.Patients (MedicalHistory) TO Receptionist;
GO
```

CLS vs. Database Views

While both CLS and standard views can restrict access to sensitive columns, they handle security differently:

Aspect	Column-Level Security (CLS)	Views
Granularity	Highly granular; different permissions can be set on different columns within the <i>same</i> table for different users.	Requires creating and managing multiple separate views for each unique set of user permissions.
Maintenance	Low maintenance; permissions are tied to the columns	Higher maintenance; views must be manually rewritten if

	themselves and adapt automatically to table changes.	the underlying table structure changes.
Performance	Highly efficient because the database engine operates directly on the base table data.	Can introduce performance overhead, especially with large tables or complex view definitions.
Transparency	Completely seamless; users query the original table normally, and the database engine handles the filtering.	Users must explicitly know to query a different database object (the view instead of the table).
Flexibility	Less flexible; strictly used for hiding columns.	

Unit 4 Configure SQL granular permissions using T-SQL

Core Permission Syntax

Database access is controlled using standard T-SQL security statements.

- **GRANT**: Gives a user or role specific permissions to an object.
- **DENY**: Explicitly blocks a user or role from an object.
- **The Override Rule**: A **DENY** statement **always supersedes** a **GRANT**. If a user is both granted and denied access to the same object, they will be blocked.

Object Permissions Breakdown

1. Tables and Views

Permissions for tables and views manage standard **Data Manipulation Language (DML)** operations. These can be applied to entire objects or restricted further down to specific columns:

Permission	Definition
SELECT	Allows the user to view data within the table or view.
INSERT	Allows the user to insert new data into the object.
UPDATE	Allows the user to modify existing data within the object.
DELETE	Allows the user to remove data from the object.

2. Functions and Stored Procedures

For programmable objects, permissions govern structural modifications and administrative control:

Permission	Definition
ALTER	Allows a user to change the definition or structure of the function or stored procedure.
CONTROL	Grants the user full ownership rights and all available privileges over the object.

Security Best Practices

Principle of Least Privilege

Users and applications should only be granted the minimum necessary permissions required to complete their tasks.

Example: If an application interacts with the data warehouse solely through stored procedures, it should only be granted **EXECUTE** permissions on those procedures, leaving it with zero direct access to the underlying tables.

Dynamic SQL and Security

Dynamic SQL allows T-SQL queries to be constructed programmatically at runtime. While powerful, it introduces major security risks like SQL injection.

When writing dynamic SQL inside stored procedures, inputs must always be sanitized using parameterization methods such as **sp_executesql** or the **QUOTENAME** function:

SQL

```
CREATE PROCEDURE sp_TopTenRows @tableName NVARCHAR ( 128 )
AS
BEGIN
    DECLARE @query NVARCHAR ( MAX );
    -- QUOTENAME safely quotes the table name input to prevent SQL injection
    SET @query = N 'SELECT TOP 10 * FROM ' + QUOTENAME (@tableName);

    EXEC sp_executesql @query;
END ;
```

Manage a Microsoft Fabric environment

(CI/CD) in Microsoft Fabric

(CI/CD) in Microsoft Fabric

Unit 1 Introduction

Overview: CI/CD in Microsoft Fabric

This module introduces **Lifecycle Management** in Microsoft Fabric, which is designed for data engineers and developers who need to collaborate, manage, and release data projects efficiently across different environments (from development to production).

Key Concepts Covered

- **Continuous Integration and Continuous Delivery (CI/CD):** Streamlining the process of incrementally releasing and integrating Fabric item changes.
- **Version Control Integration:** Connecting Microsoft Fabric with popular source control systems like **GitHub** and **Azure DevOps** to track changes and improve delivery processes.
- **Deployment Pipelines:** Moving changes through structured stages (such as development, test, and production workspaces) ensuring they can be tested and released quickly.
- **Automation:** Utilizing **Fabric REST APIs** to automate deployment and testing workflows.

Unit 2 Understand Continuous Integration and Continuous Delivery (CI/CD)

Core Concepts

- **Continuous Integration (CI):** The practice of frequently committing code changes to a shared branch in a version control system. This allows automated builds and testing to catch bugs and merge conflicts early, preventing delivery delays.
 - **Continuous Delivery:** The stage following CI where code is automatically deployed to a staging environment for additional testing before manual or scheduled production releases.
 - **Continuous Deployment:** An fully automated process where code updates that pass all testing stages are directly released straight into production.
-

Implementation in Microsoft Fabric

Managing the lifecycle of Fabric items relies on three distinct pillars:

- **Git Integration (Integration):** Provides version control and history tracking, allowing development teams to collaborate safely using isolated branches before merging code.
- **Deployment Pipelines (Deployment):** Used to progress and promote validated code changes across structured environments (typically **Development** $\rightarrow$ **Test** $\rightarrow$ **Production** workspaces).
- **Fabric REST APIs (Automation):** Enables developers to programmatically automate and manage the entire integration and deployment workflow

Unit 3: Implement version control and Git integration from the Microsoft Fabric CI/CD training module:

Core Concepts

- **Version Control in Fabric:** Integration occurs at the **workspace level**. It allows developers to track revisions, collaborate safely, and roll back code changes using **GitHub** or **Azure DevOps**.
- **Isolated Development:** Working directly in a shared, live workspace overrides other users' work. Best practice dictates using an isolated environment (a separate workspace or local client tools like Power BI Desktop and VS Code) to develop features independently.

Connection and Synchronization

- **Connecting to Git:** You connect a Fabric workspace to a specific repository and branch via the **Git integration** section in the workspace settings.
 - **Tracking Changes:** Once connected, a Git status column appears in the workspace. A source control icon displays the number of differing items between the workspace and the remote branch.
 - **Syncing Workflows:** * Use **Changes** to push updates from your Fabric workspace to the Git branch.
 - Use **Updates** to pull new commits from the Git branch into your Fabric workspace.
-

Branching & Deployment Scenarios

To ensure safe feature development, you can use one of two isolated workflows before merging changes into the main branch:

1. Web-Based (Fabric Workspace) Workflow

1. Connect a shared development workspace to the **main** branch.
2. Use **Branch out to new workspace** to automatically spin up a dedicated branch and an isolated workspace copy.
3. Make and commit changes within the isolated workspace.
4. Create a **Pull Request (PR)** in Git to merge your feature branch back into **main**.
5. Sync the shared development workspace to pull in the newly approved changes.

2. Client-Tool (Local) Workflow

1. Clone the repository to your local machine to work via Power BI Desktop or VS Code.
2. Push local changes to a remote feature branch and test them in an isolated workspace.
3. Submit a **PR** in Git to merge updates into the **main** branch.
4. Open the shared development workspace and accept the prompt to sync changes from the repository.

Unit 4: Implement deployment pipelines

Core Concepts

- **Deployment Pipelines:** An automated way to manage the lifecycle of Fabric content by cloning and promoting items across isolated environment stages (typically **Development** $\rightarrow$ **Test** $\rightarrow$ **Production** workspaces). This ensures updates are thoroughly tested before reaching end users.
- **Creation Methods:** You can build a new pipeline either from the **Workspaces** icon on the left navigation pane or by selecting the **Create deployment pipeline** icon at the top of an active workspace.

Deployment Workflow

- **Workspace Assignment:** During setup, you define your stages and assign a specific Fabric workspace to each stage.
 - **Promoting Content:** Moving content is a simple click-to-deploy process. When you deploy from one stage to another (e.g., Development to Test), Fabric automatically clones all workspace items into the target environment.
-

Combining Pipelines with Git Integration

To avoid synchronization conflicts across multiple environments, Microsoft highlights a common, clean workflow approach:

1. **Connect Only Development:** Link *only* your **Development** workspace to your Git repository branch for version control.
2. **Commit via Git:** Make and track code changes purely within the Development stage using the Fabric source control panel.
3. **Deploy via Pipelines:** Use Fabric's native deployment pipeline buttons to promote those validated updates to the **Test** and **Production** environments, keeping all stages cleanly synchronized within Fabric.

 **Tip:** For alternative setups and deeper architectural options, check out Microsoft's guide on how to [Choose the best Fabric CI/CD workflow option for you](#).

Unit 5: Automate CI/CD using Fabric APIs

Core Concepts

- **Fabric REST APIs:** Representational State Transfer Application Programming Interfaces allow developers to programmatically interact with and manage Fabric procedures.
 - **Key Advantages:** * **Consistency:** Eliminates human error by automating repetitive, ongoing data processing routines.
 - **Integration:** Seamlessly connects Fabric data pipelines with external third-party applications and deployment orchestration tools.
-

What You Can Automate

The Fabric CI/CD REST APIs are divided into two primary categories to handle automation tasks:

1. Git Integration APIs

- **Commit Changes:** Automatically push updates made inside a Fabric workspace directly to a connected remote Git branch.
- **Pull Updates:** Programmatically update a workspace with the latest commits pushed to its connected Git branch.
- **Check Status:** Use the Git status API to identify which workspace items have incoming changes or uncommitted modifications.

2. Deployment Pipelines APIs

- **List Stage Items:** Query and return supported items residing in a workspace that is assigned to a specific deployment pipeline stage.
- **Deploy Stage Content:** Programmatically execute a deployment to clone and promote content from one pipeline stage to the next.

Monitor activities in Microsoft Fabric

Monitor activities in Microsoft Fabric

UNIT 1 Introduction

Overview

In data analytics, raw data must be collected, combined, ingested, and transformed to become useful for business decision-making. Because errors can disrupt users downstream, monitoring these background ingestion and transformation processes is critical.

Key Learning Objectives

By the end of this module, you will understand how to:

- **Use Monitor Hub:** Track and manage data ingestion and data transformation activities across your Fabric environment.
- **Utilize Activator:** Automatically trigger actions and alerts when specific patterns or shifting conditions are detected in your changing data.
- **Gain Practical Experience:** Apply these concepts through a hands-on exercise using the Monitor Hub.

UNIT 2 Understand monitoring

Core Definition

Monitoring is the continuous process of collecting system data and metrics to verify system health, track performance, and expose errors. Analyzing historical data helps establish baseline behaviors, investigate issues, and quickly remediate errors to minimize downtime.

Fabric Activities to Monitor

Because data processes have complex dependencies, you must manage end-to-end reliability by monitoring these specific Microsoft Fabric components:

- **Data Pipeline Activities:** Tracking the overall success or failure of ETL (Extract, Transform, Load) jobs and comparing current performance against job history to pinpoint when errors were introduced.
- **Dataflows:** Monitoring start/end times, execution duration, and table load status within this low-code ingestion interface.

- **Semantic Model Refreshes:** Watching for data model refresh statuses and identifying transient issues by tracking retry attempts before marking an execution as a total failure.
 - **Spark Jobs, Notebooks, and Lakehouses:** Checking Apache Spark job progress, resource utilization, task execution, and system logs.
 - **Eventstreams:** Monitoring streaming event data, ingestion performance, and system status for real-time analytics.
-

Monitoring Best Practices

To keep data ingestion and transformation running smoothly, the guide highlights five core practices:

1. **Identify what to monitor** and define clear metrics to track.
2. **Collect and analyze data regularly** to build a profile of normal system behavior.
3. **Review logs and metrics** frequently to make it easier to spot anomalies.
4. **Take immediate action** to troubleshoot deviations from standard baselines.
5. **Optimize performance** by utilizing monitoring insights to eliminate processing bottlenecks.

Unit 3 Use Microsoft Fabric Monitor Hub

Core Definition

The **Monitor Hub** is a centralized visualization tool within Microsoft Fabric. Instead of monitoring individual processes separately, it aggregates and displays tracking data from multiple Fabric items in a single, common interface to help you easily spot trends or anomalies.

Tracked Activities

The Monitor Hub displays execution and refresh metadata for various Fabric operations, including:

- Data pipeline execution histories
 - Dataflow executions
 - Datamart and semantic model refreshes
 - Apache Spark job details and notebook execution histories
-

Key Functionalities

1. Viewing Activity Details

By selecting *Monitor* from the Fabric navigation pane, you can see a list of ongoing or completed operations. Hovering over an activity and clicking the **ellipsis (...)** allows you to perform context-specific actions such as:

- Opening the activity or viewing historical runs
- Retrying a failed execution
- Inspecting custom metadata (e.g., activity status, start/end times, duration, and activity statistics)

2. Deep-Dive Investigation

For deeper troubleshooting, many activities feature hyperlinks that let you drill down into specific execution details. This includes tracking errors, verifying success/failure points, and analyzing Spark applications triggered across Notebooks, Pipelines, and Spark Job Definitions.

Unit 4 Take action with Microsoft Fabric Activator

Core Definition

[Activator](#) is a tool within Microsoft Fabric's Real-Time Intelligence that automates event processing. It monitors streaming data, detects anomalies or specific patterns, and automatically triggers specified actions without requiring manual oversight.

Key Concepts

Activator functions using a framework built on four core components:

- **Events:** Individual point-in-time data records within a continuous data stream (e.g., a single reading from an IoT sensor).
- **Objects:** The business entity that the data represents (e.g., a specific sensor, sales order, or shipment).
- **Properties:** The specific data fields mapped to an object that reflect its current state (e.g., temperature values or total sales amounts).

- **Rules:** The logic or thresholds defined by users (which can include Kusto Query Language, or KQL, queries). When a property value triggers a rule, an automated action is executed.
-

Triggered Actions & Common Use Cases

When a rule's conditions are met, Activator can execute several automated responses, such as alerting users via email, kicking off Power Automate workflows, or running Fabric job items like data pipelines and Spark-based notebooks.

Typical scenarios for deploying Activator include:

- **Infrastructure & Data Workflows:** Responding immediately to data processing failures or anomalies.
- **IoT & Operations:** Tracking temperature shifts to protect perishable goods, or flagging failing hardware (like grocery store freezers) before damage occurs.
- **Business & Marketing:** Alerting store managers when a shipment is delayed, or automatically initiating marketing campaigns and advertisements when product sales drop below a certain baseline.

Secure data access in Microsoft Fabric

Unit 1 Introduction

- **Layered Security Model:** Microsoft Fabric allows administrators to control data access at multiple levels—ranging from an entire workspace down to individual tables and folders.
- **The Principle of Least Privilege:** The module uses a healthcare company scenario to demonstrate how to grant different roles (e.g., data engineers, analysts, and report consumers) the exact level of access they need without exposing sensitive data.
- **Core Concepts Taught:** * How **workspace roles** and **item permissions** control access to Fabric items.
 - How to use **T-SQL permissions** and **OneLake security roles** for granular, fine-grained control over the actual data within those items.

Unit 2 Understand the Fabric security mode

Three Levels of Access Evaluation

Microsoft Fabric sequentially checks permissions across three distinct layers to enforce the principle of least privilege:

1. **Microsoft Entra ID authentication:** Verifies if the user can log in.
2. **Fabric access:** Determines if the user is allowed to access the Fabric environment.
3. **Data security:** Validates if the user can perform specific actions on individual tables or files.

The Four Data Security Controls

The final data security layer relies on four cascading access controls that can work together:

- **Workspace Roles:** Provides broad, workspace-wide permissions. Use this when a user needs to collaborate on all items within a workspace.
- **Item Permissions:** Restricts access to a single specific item (like a standalone lakehouse or warehouse) rather than the whole workspace.
- **Compute Permissions:** Controls what a user can do through a specific query engine (such as SQL analytics endpoints or semantic models) using permissions like Read or Write.
- **OneLake Security:** Provides the most granular control *within* an item by restricting access to specific tables or folders. These rules apply universally across all compute engines (Spark, SQL, and OneLake APIs).

Unit 3 Configure workspace and item permissions

1. Workspace Roles (Broad Access)

Workspace roles apply to **all items** within a given workspace. They are best used when a user needs broad access to collaborate. There are four roles:

- **Admin:** Full control over all content, data, and permissions.
- **Member:** Can create, view, modify, and share all content and data.
- **Contributor:** Can create, view, and modify content, but cannot share it or manage permissions.
- **Viewer:** Can view content but cannot modify it. *(Note: Viewers can see items listed, but do not have access to the underlying OneLake data by default).*

2. Item Permissions (Targeted Access)

When a user only needs access to a specific asset (like a single lakehouse) rather than everything in the workspace, you use **Item Permissions**. This aligns with the principle of least privilege by removing them from the workspace role and sharing the item directly.

When sharing a **Lakehouse**, the **Read** permission is always granted (allowing the user to see metadata and associated reports). You can then selectively layer on additional data access permissions:

Additional Permission 1	What it Allows 2
Read all SQL endpoint data	Read data from the SQL analytics endpoint using T-SQL.
Read all Apache Spark and subscribe to events	Read lakehouse data through Apache Spark and OneLake APIs (automatically adds the user to the <i>DefaultReader</i> OneLake security role).
Build reports on the default semantic model	Create Power BI reports on the default semantic model.

Unit 4 Apply granular permissions

When item permissions are too broad (e.g., granting access to an entire lakehouse when a user only needs specific tables or rows), you must apply **granular permissions**. The method depends on how the user accesses the data:

1. Granular Permissions via T-SQL (Compute Engine)

When users query lakehouse data through the **SQL analytics endpoint**, you can use standard Data Control Language (DCL) commands to restrict access:

- **Commands:** Use **GRANT**, **DENY**, and **REVOKE** at the table or view level.
 - **Advanced Security:** You can also configure **Row-level security**, **Column-level security**, and **Dynamic data masking**.
-

2. Granular Permissions via OneLake Security (Universal)

OneLake security uses a **Role-Based Access Control (RBAC)** model. These roles are enforced universally across *all* Fabric compute engines (Spark, SQL, and OneLake APIs).

- **Who it affects:** It targets users with the **Viewer** workspace role or those with only **Read** item permissions (who otherwise have no OneLake data access by default).
- **Who can manage it:** Only workspace **Admins** and **Members** can create or modify these roles.

Every custom OneLake security role consists of four components:

- **Data:** The specific tables or folders the role can access.
- **Permission:** The level of access allowed (**Read** or **ReadWrite**).
- **Members:** The users or Azure AD groups assigned to the role.
- **Constraints:** Optional row or column filters to further restrict data visibility.

 **Important Note on the DefaultReader Role** Every new lakehouse automatically includes a **DefaultReader** role that grants full read access to all data. When you assign a user to a restricted custom role, you **must explicitly remove them from the DefaultReader role**, or they will retain full read access.